UNIVERSIDADE PAULISTA – UNIP

Projeto Integrado Multidisciplinar

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

RENAM DA SILVA DANTAS – RA R311JG2

GABRIEL EDUARDO CARNEIRO CLEMENTE – RA H918790

DIOGO DOS SANTOS ANJOS – RA R2336A3

VITOR FAUSTINO ARAÚJO DE LIMA – RA H86DBJ2

BERNARDO COELHO DO AMARAL SILVA – RA H9629B2

FELIPE PEREIRA FERNANDES – RA R357511

VINICIUS MATHEUS DE SOUZA ALVES – RA R2581F0

SABOR & CIA BISTRÔ

Cardápio digital acessível com controle de disponibilidade por filial

Barueri

2026

RENAM DA SILVA DANTAS – RA R311JG2
GABRIEL EDUARDO CARNEIRO CLEMENTE – RA H918790
DIOGO DOS SANTOS ANJOS – RA R2336A3
VITOR FAUSTINO ARAÚJO DE LIMA – RA H86DBJ2
BERNARDO COELHO DO AMARAL SILVA – RA H9629B2
FELIPE PEREIRA FERNANDES – RA R357511
VINICIUS MATHEUS DE SOUZA ALVES – RA R2581F0

SABOR & CIA BISTRÔ

Cardápio digital acessível com controle de disponibilidade por filial

Projeto Integrado Multidisciplinar em Análise e Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para obtenção do título de tecnólogo em Análise e Desenvolvimento de Sistemas, apresentado à Universidade Paulista – UNIP.

Orientador(a): Prof. Edy Hayashida

Barueri
2026

RESUMO

Este trabalho apresenta o projeto de um cardápio digital acessível, com controle de disponibilidade de pratos por unidade, para a rede fictícia de restaurantes Sabor & Cia Bistrô. O problema investigado é a defasagem entre o que o cliente vê no cardápio e o que está efetivamente disponível em cada filial, agravada pela baixa acessibilidade dos cardápios em PDF e pela ausência de informação estruturada sobre alérgenos. A pesquisa, de natureza aplicada e exploratória, com abordagem qualitativa e procedimentos bibliográfico e documental, integra seis componentes curriculares: pesquisa, tecnologia e inovação; engenharia de software, modelagem de banco de dados, programação estruturada, redes de computadores e sistemas distribuídos, e direitos humanos, inclusão e segurança. Foram elaborados os requisitos e a modelagem UML da solução, os modelos conceitual, lógico e físico do banco de dados relacional em MySQL — com discussão do uso de um banco NoSQL para o histórico de alterações —, uma simulação em linguagem C da lógica central de disponibilidade e o projeto da infraestrutura de rede das unidades e da arquitetura distribuída em nuvem. A solução foi avaliada quanto à acessibilidade (WCAG 2.2 e ABNT NBR 17225), à inclusão digital, à proteção de dados (LGPD) e aos direitos do consumidor, resultando em uma proposta bilíngue, responsiva e centrada na privacidade.

Palavras-chave: Cardápio digital. Acessibilidade. Restaurantes. Sistemas distribuídos. LGPD.

ABSTRACT

This paper presents the design of an accessible digital menu, with per-branch dish availability control, for the fictitious restaurant chain Sabor & Cia Bistrô. The investigated problem is the gap between what the customer sees on the menu and what is actually available at each branch, worsened by the poor accessibility of PDF menus and by the lack of structured allergen information. The research is applied and exploratory, with a qualitative approach and bibliographic and documentary procedures, and integrates six academic subjects: research, technology and innovation; software engineering, database modeling, structured programming, computer networks and distributed systems, and human rights, inclusion and security. The requirements and the UML modeling were produced, along with the conceptual, logical and physical models of the relational MySQL database — with a discussion of using a NoSQL database for the change history —, a C-language simulation of the core availability logic, and the design of the branches' network infrastructure and the distributed cloud architecture. The solution was assessed regarding accessibility (WCAG 2.2 and ABNT NBR 17225), digital inclusion, data protection (LGPD) and consumer rights, resulting in a bilingual, responsive and privacy-centered proposal.

Keywords: Digital menu. Accessibility. Restaurants. Distributed systems. LGPD.

LISTA DE FIGURAS

Figura 1 – Diagrama de casos de uso 20
Figura 2 – Diagrama de classes 21
Figura 3 – Diagrama de atividades – alteração de disponibilidade 22
Figura 4 – Arquitetura do sistema em camadas 23
Figura 5 – Modelo conceitual (entidade-relacionamento) 24
Figura 6 – Modelo lógico (DER) 25
Figura 7 – Fluxograma – exibição do cardápio da filial 32
Figura 8 – Fluxograma – alteração de disponibilidade 32
Figura 9 – Topologia geral (WAN, VPN e nuvem) 35
Figura 10 – Topologia da rede local de uma filial 36

LISTA DE QUADROS

Quadro 1 – Trabalhos relacionados 16
Quadro 2 – Product backlog (itens priorizados) 19
Quadro 3 – Plano de VLANs e sub-redes de uma filial (Jardins) 37

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface
CDN	Content Delivery Network
DER	Diagrama Entidade-Relacionamento
HTML	HyperText Markup Language
HTTPS	Hypertext Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
IP	Internet Protocol
JSON	JavaScript Object Notation
LAN	Local Area Network
LGPD	Lei Geral de Proteção de Dados Pessoais
NBR	Norma Brasileira
NoSQL	Not Only SQL
PDF	Portable Document Format
PoE	Power over Ethernet
QR	Quick Response
REST	Representational State Transfer
RF	Requisito Funcional
RNF	Requisito Não Funcional
SQL	Structured Query Language
TLS	Transport Layer Security
UML	Unified Modeling Language
UTM	Unified Threat Management
VLAN	Virtual Local Area Network
VPC	Virtual Private Cloud
VPN	Virtual Private Network
WAF	Web Application Firewall
WAN	Wide Area Network
WCAG	Web Content Accessibility Guidelines

SUMÁRIO

1 INTRODUÇÃO 9
2 CARACTERIZAÇÃO DA ORGANIZAÇÃO 10
2.1 Identificação e ramo de atuação 10
2.2 Missão 10
2.3 Principais produtos e serviços 10
2.4 Estrutura organizacional 10
2.5 Principais processos de negócio 11
2.6 Público-alvo 11
2.7 Problemas identificados 11
2.8 Justificativa da solução proposta 12
3 PESQUISA CIENTÍFICA E INOVAÇÃO 13
3.1 Definição do problema 13
3.2 Objetivos 13
3.2.1 Objetivo geral 13
3.2.2 Objetivos específicos 13
3.3 Metodologia 14
3.4 Fundamentação bibliográfica 14
3.4.1 Cardápios digitais e adoção de QR code 14
3.4.2 Acessibilidade digital 15
3.4.3 Inclusão digital 15
3.4.4 Informação ao consumidor e alergias alimentares 15
3.5 Análise de trabalhos relacionados 16
3.6 Justificativa científica 16
3.7 Diferenciais inovadores 17
4 ENGENHARIA DE SOFTWARE 18
4.1 Levantamento e especificação de requisitos 18
4.2 Planejamento com o framework Scrum 19
4.2.1 Backlog do produto 19
4.3 Modelagem da solução em UML 20
4.3.1 Diagrama de casos de uso 20
4.3.2 Diagrama de classes 20
4.3.3 Diagrama de atividades e fluxogramas 21
4.4 Escopo do produto e da simulação 22
4.5 Arquitetura do sistema 22
5 MODELAGEM DE BANCO DE DADOS 24

5.1 Modelo conceitual 24
5.2 Modelo lógico 25
5.3 Modelo físico em MySQL 26
5.4 Banco NoSQL para o histórico de alterações 26
5.5 Integridade, transações e segurança dos dados 28
5.6 Principais comandos SQL 28
6 PROGRAMAÇÃO ESTRUTURADA 30
6.1 Estruturas de dados 30
6.2 Organização em funções 30
6.3 Funcionalidades implementadas 30
6.4 Persistência em arquivos 31
6.5 Validação de entradas e robustez 31
6.6 Fluxogramas 31
6.7 Segurança: simulação acadêmica e implementação prevista 32
7 REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS 34
7.1 Arquitetura distribuída em nuvem 34
7.2 Topologia WAN e interligação das unidades 35
7.3 Rede local de uma filial 35
7.4 Propriedades de sistemas distribuídos 36
7.5 Endereçamento IP e escalabilidade 37
8 DIREITOS HUMANOS, INCLUSÃO E SEGURANÇA 38
8.1 Acessibilidade digital 38
8.2 Inclusão digital 38
8.3 Informação ao consumidor e alérgenos 38
8.4 Proteção de dados e privacidade (LGPD) 39
8.5 Direitos humanos e responsabilidade social 39
8.6 Aspectos éticos e impactos sociais 39
9 INTEGRAÇÃO DA SOLUÇÃO 40
9.1 Visão integrada do funcionamento 40
9.2 Integração entre as disciplinas 40
9.3 Da simulação ao sistema em produção 40
9.4 Benefícios esperados e limitações 41
10 CONCLUSÃO 42
REFERÊNCIAS 43

1 INTRODUÇÃO

Este trabalho apresenta o projeto de um cardápio digital acessível, com controle de disponibilidade por unidade, para a rede fictícia de restaurantes Sabor & Cia Bistrô, criada para fins acadêmicos. A organização opera três unidades na cidade de São Paulo, com cardápio e preços padronizados, e enfrenta dificuldades para manter o cliente informado sobre o que está efetivamente disponível em cada unidade.

O objetivo do trabalho é propor uma solução computacional que torne o cardápio acessível e mantenha a disponibilidade dos pratos atualizada por filial, integrando os conhecimentos das disciplinas do semestre. Para isso, foram elaborados os requisitos e a modelagem da solução, o projeto do banco de dados, uma simulação da lógica central em linguagem C, o projeto da infraestrutura de rede e da arquitetura em nuvem e a avaliação da solução quanto à acessibilidade, à inclusão e à segurança.

Metodologicamente, a pesquisa é aplicada e exploratória, de abordagem qualitativa, apoiada em levantamento bibliográfico e documental. O desenvolvimento foi planejado com o framework Scrum e organizado em etapas incrementais, descritas na seção 3.3.

O documento está organizado da seguinte forma: o capítulo 2 caracteriza a organização; o capítulo 3 apresenta a pesquisa científica e os diferenciais inovadores; o capítulo 4 trata da engenharia de software; o capítulo 5, da modelagem de banco de dados; o capítulo 6, da programação estruturada; o capítulo 7, das redes e sistemas distribuídos; o capítulo 8, dos direitos humanos, inclusão e segurança; o capítulo 9, da integração da solução; e o capítulo 10 apresenta a conclusão.

2 CARACTERIZAÇÃO DA ORGANIZAÇÃO

A organização apresentada neste trabalho é fictícia e foi criada para fins acadêmicos, conforme orienta o Manual do PIM II. Ainda assim, sua caracterização procura reproduzir as condições de uma rede real de restaurantes da cidade de São Paulo, para que os problemas identificados e a solução proposta sejam plausíveis.

2.1 Identificação e ramo de atuação

A Sabor & Cia Ltda., que opera com o nome fantasia Sabor & Cia Bistrô, atua no ramo de alimentação fora do lar, no segmento de bistrô contemporâneo de padrão médio-alto. A empresa mantém uma matriz administrativa em Alphaville, no município de Barueri (SP), que não atende ao público, e três restaurantes na cidade de São Paulo, localizados nos bairros Jardins, Itaim Bibi e Pinheiros.

A separação entre razão social e nome fantasia é intencional: a pessoa jurídica responde pelas obrigações legais e fiscais do grupo, enquanto a marca comunicada ao cliente precisa refletir o posicionamento dos restaurantes.

2.2 Missão

A missão da Sabor & Cia Bistrô é oferecer uma experiência gastronômica de qualidade e consistente nas três unidades, com atendimento acolhedor e transparência sobre os ingredientes de cada prato.

2.3 Principais produtos e serviços

A rede trabalha com cardápio fixo, idêntico nas três unidades e com o mesmo preço em todas elas, composto por entradas, pratos principais, sobremesas e uma carta de vinhos servidos em taça ou garrafa. Os restaurantes funcionam no almoço e no jantar, e o ticket médio é de R$ 125,00 por pessoa. O pedido é lançado pelo garçom em uma maquininha de cartão, que também é usada para o pagamento na própria mesa.

2.4 Estrutura organizacional

A estrutura divide-se em dois níveis. Na matriz ficam a diretoria, formada pelos sócios, e as áreas administrativo-financeira, de compras e de marketing; o suporte de tecnologia da informação é terceirizado. Cada restaurante conta com um gerente,

responsável pela operação diária, um chef e sua equipe de cozinha, um maître e a equipe de garçons.

As decisões sobre o cardápio, os preços e a identidade da marca são centralizadas na matriz, enquanto a gestão do dia a dia, incluindo o que está ou não disponível para venda, cabe a cada unidade. É justamente na fronteira entre essas duas responsabilidades que surgem os problemas descritos na seção 2.7.

2.5 Principais processos de negócio

Os processos centrais da organização são os seguintes:

a) atendimento ao cliente: o maître recebe o cliente e apresenta o cardápio impresso; o garçom lança o pedido na maquininha, a cozinha prepara os pratos e o pagamento é feito na mesa;

b) definição do cardápio: a matriz define os pratos, as descrições e os preços, que valem para as três unidades;

c) controle diário de disponibilidade: quando um prato se esgota, o chef avisa o gerente, que repassa a informação verbalmente aos garçons;

d) compras: a matriz consolida as necessidades das unidades e negocia com os fornecedores.

2.6 Público-alvo

O público da rede é formado por adultos das classes A e B: executivos no horário de almoço, casais e grupos no jantar e turistas nacionais e estrangeiros, presença mais frequente nas unidades Jardins e Itaim Bibi.

2.7 Problemas identificados

A análise dos processos permitiu identificar os seguintes problemas:

a) o cliente escolhe um prato que já se esgotou naquela unidade e só descobre isso depois de fazer o pedido, o que gera frustração e retrabalho;

b) a comunicação sobre pratos esgotados depende de avisos verbais entre cozinha, gerente e salão, que falham nos horários de maior movimento; além disso, a maquininha continua permitindo lançar um item indisponível;

c) o cardápio impresso não comporta informações sobre ingredientes, alérgenos e harmonização, de modo que clientes com alergia alimentar e turistas dependem inteiramente do garçom;

d) o cardápio impresso não pode ser lido de forma autônoma por uma pessoa com deficiência visual;

e) a matriz não tem uma visão consolidada de quais pratos se esgotam, em qual unidade e com que frequência, informação que seria útil para planejar as compras.

2.8 Justificativa da solução proposta

Os problemas descritos têm uma causa comum: a informação sobre o que está disponível em cada unidade não circula de forma confiável entre a cozinha, o salão, as maquininhas e o cliente. A solução proposta é um cardápio digital centralizado, acessado pelo cliente por meio de um QR code (Quick Response code) próprio de cada unidade, no qual a matriz mantém um cardápio único e cada gerente ativa ou desativa os pratos de sua unidade em tempo real. A mesma informação é consumida pelas maquininhas dos garçons, que deixam de oferecer itens esgotados.

O cardápio digital não substitui o atendimento: o cardápio impresso e o garçom continuam presentes, e o recurso digital acrescenta o que o papel não comporta, como ingredientes, alérgenos, harmonização, versão em inglês e compatibilidade com leitores de tela. A fundamentação dessa escolha é apresentada no capítulo 3.

3 PESQUISA CIENTÍFICA E INOVAÇÃO

3.1 Definição do problema

O uso de cardápios acessados por QR code se difundiu no setor de alimentação durante a pandemia de COVID-19. Pesquisa da Associação Brasileira de Bares e Restaurantes ([2023]), realizada no segundo semestre de 2022, indicou que 38% dos estabelecimentos adotavam esse tipo de cardápio, 25% estavam implantando ou pretendiam implantar e 11% haviam adotado e desistido após o fim das restrições sanitárias. A tecnologia, portanto, já é conhecida do setor, mas sua permanência depende do valor que ela agrega ao cliente e à operação.

Na prática, muitos desses cardápios são apenas a versão em PDF (Portable Document Format) do cardápio impresso: não informam o que está disponível naquele momento, não trazem informações adicionais e, em geral, não são adequados a leitores de tela. Diante disso, o problema de pesquisa deste trabalho é: **como uma rede de restaurantes com várias unidades pode manter o cardápio apresentado ao cliente sincronizado com a disponibilidade real de cada unidade, garantindo acesso à informação a pessoas com deficiência, a clientes com alergias alimentares e a estrangeiros?**

3.2 Objetivos

3.2.1 Objetivo geral

Propor uma solução computacional de cardápio digital acessível, com controle de disponibilidade por unidade e integração às maquininhas dos garçons, para a rede fictícia Sabor & Cia Bistrô.

3.2.2 Objetivos específicos

a) levantar os requisitos da solução e planejar seu desenvolvimento com o framework Scrum;

b) modelar o banco de dados relacional e discutir o uso de um banco NoSQL (Not Only SQL) para o histórico de alterações;

c) implementar em linguagem C a lógica central do controle de disponibilidade;

d) projetar a infraestrutura de rede das unidades e a arquitetura distribuída em nuvem;

e) avaliar a solução quanto à acessibilidade, à inclusão digital, à proteção de dados e aos direitos humanos.

3.3 Metodologia

Quanto à finalidade, a pesquisa é aplicada, pois busca resolver um problema concreto de uma organização; quanto aos objetivos, é exploratória, já que procura proporcionar maior familiaridade com o problema e torná-lo mais explícito (Gil, 2022). Quanto aos procedimentos, combina pesquisa bibliográfica, baseada em livros e artigos científicos, com pesquisa documental, que abrange leis, normas técnicas e relatórios estatísticos de órgãos oficiais (Marconi; Lakatos, 2021). A abordagem é qualitativa.

O trabalho foi desenvolvido nas seguintes etapas: levantamento bibliográfico e documental; caracterização da organização fictícia e de seus processos; levantamento de requisitos e modelagem do sistema com a UML (Unified Modeling Language); modelagem do banco de dados; implementação de uma simulação em linguagem C; projeto da infraestrutura de rede; e análise da solução à luz da legislação de acessibilidade e de proteção de dados. Por se tratar de uma organização fictícia, não houve coleta de dados com usuários reais, o que constitui uma limitação discutida no capítulo 9.

3.4 Fundamentação bibliográfica

3.4.1 Cardápios digitais e adoção de QR code

A literatura internacional sobre cardápios por QR code concentra-se na aceitação da tecnologia pelos consumidores. Em estudo com 844 clientes de restaurantes nos Estados Unidos, Iskender *et al.* (2024) mostraram que a intenção de uso é explicada principalmente pela possibilidade de experimentar a tecnologia e pela percepção de higiene. Koay e Ang (2024), por sua vez, verificaram que a expectativa de desempenho, a facilidade de uso, a influência social, o hábito e a percepção de proteção da privacidade influenciam positivamente a intenção de usar esses cardápios após a pandemia.

Esses resultados indicam que um cardápio digital precisa ser simples de usar e transmitir segurança quanto aos dados do cliente, dois requisitos incorporados à solução proposta.

3.4.2 Acessibilidade digital

Segundo o Censo Demográfico 2022, o Brasil tem 14,4 milhões de pessoas com deficiência, o equivalente a 7,3% da população com 2 anos ou mais; a dificuldade mais frequente é a de enxergar, relatada por 7,9 milhões de pessoas (Instituto Brasileiro de Geografia e Estatística, 2025). Para esse público, um cardápio impresso ou em imagem exige a ajuda de outra pessoa, enquanto um cardápio digital bem construído pode ser lido de forma autônoma com um leitor de tela.

A Lei Brasileira de Inclusão determina, em seu art. 63, que é obrigatória a acessibilidade nos sítios da internet mantidos por empresas com sede ou representação comercial no país, conforme as melhores práticas adotadas internacionalmente (Brasil, 2015). A principal referência internacional é a WCAG (Web Content Accessibility Guidelines) 2.2, que organiza os requisitos em quatro princípios: o conteúdo deve ser perceptível, operável, compreensível e robusto (W3C, 2023). No Brasil, a ABNT NBR 17225, publicada em 2025 e alinhada à WCAG 2.2, estabelece os requisitos de acessibilidade para conteúdo e aplicações web (Associação Brasileira de Normas Técnicas, 2025).

3.4.3 Inclusão digital

Um cardápio acessível apenas pelo celular pode excluir parte do público. A pesquisa TIC Domicílios 2025 estimou 28 milhões de brasileiros que não usam a internet, dos quais 16 milhões têm 60 anos ou mais (Comitê Gestor da Internet no Brasil, 2025). Por essa razão, a solução mantém o cardápio impresso e o atendimento pelo garçom, tratando o recurso digital como complemento.

3.4.4 Informação ao consumidor e alergias alimentares

O Código de Defesa do Consumidor assegura, em seu art. 6º, inciso III, o direito à informação adequada e clara sobre os produtos, incluindo sua composição e os riscos que apresentam, e determina, no parágrafo único do mesmo artigo, que essa informação seja acessível à pessoa com deficiência (Brasil, 1990). No caso das alergias alimentares, essa informação é uma questão de saúde: o Consenso Brasileiro

sobre Alergia Alimentar relata estudo com adultos brasileiros no qual 10,8% referiam sintomas de alergia alimentar, embora apenas 1,0% tivessem diagnóstico médico (Solé *et al.*, 2018).

3.5 Análise de trabalhos relacionados

O Quadro 1 resume os trabalhos que mais se aproximam da proposta deste projeto, indicando a contribuição de cada um e a lacuna que permanece em aberto.

Quadro 1 – Trabalhos relacionados

Trabalho	**Foco**	**Principal contribuição**	**Lacuna em relação a este projeto**
Iskender, Sirakaya-Turk e Cardenas (2023)	Percepção de clientes sobre cardápios por QR code (estudo qualitativo)	Clientes valorizam rapidez e ausência de contato, mas criticam cardápios pouco informativos e em PDF mal projetado	Não trata de acessibilidade nem de disponibilidade em tempo real
Iskender *et al.* (2024)	Modelo de aceitação de cardápios por QR code (844 clientes)	Experimentação e higiene explicam a intenção de uso	Foco na aceitação, não no conteúdo do cardápio
Koay e Ang (2024)	Intenção de uso no pós-pandemia (200 respondentes)	Facilidade de uso e percepção de privacidade influenciam a adoção	Não considera pessoas com deficiência
Radke *et al.* (2016)	Alergia alimentar em 278 restaurantes dos EUA	Apenas 22% dos cardápios mencionavam alérgenos	Não propõe solução tecnológica

Fonte: elaborado pelos autores (2026).

Iskender, Sirakaya-Turk e Cardenas (2023) mostram que as críticas dos clientes recaem mais sobre a qualidade da informação do que sobre a tecnologia em si. Radke *et al.* (2016) evidenciam que a informação sobre alérgenos nos cardápios é, em geral, precária. Os demais trabalhos concentram-se em explicar por que o cliente aceita ou rejeita o cardápio digital.

3.6 Justificativa científica

A análise da literatura mostra dois conjuntos de estudos que não se comunicam: os que tratam da aceitação do cardápio digital e os que tratam da informação sobre alérgenos. Nenhum dos trabalhos analisados integra, em uma mesma solução, a acessibilidade para pessoas com deficiência, a informação estruturada sobre alérgenos, o cardápio bilíngue e a disponibilidade atualizada por unidade. Essa lacuna justifica a proposta, que também responde a exigências legais

concretas: a acessibilidade de sítios empresariais (Brasil, 2015) e o direito do consumidor à informação acessível (Brasil, 1990).

3.7 Diferenciais inovadores

Em relação às soluções encontradas na literatura e no mercado, a proposta apresenta os seguintes diferenciais:

a) disponibilidade por unidade atualizada em até 30 segundos e compartilhada entre o cardápio do cliente e as maquininhas dos garçons;

b) cardápio construído em HTML (HyperText Markup Language) estruturado, e não em PDF, seguindo a WCAG 2.2 nível AA e a ABNT NBR 17225;

c) alérgenos cadastrados de forma estruturada, com filtro por alérgeno e aviso de que a informação é de referência;

d) versões em português e inglês, com o idioma marcado no código da página para que o leitor de tela faça a pronúncia correta;

e) privacidade desde a concepção: o cliente não se cadastra e o cardápio não utiliza cookies de rastreamento;

f) papel de complemento ao atendimento, mantendo o cardápio impresso e o garçom.

4 ENGENHARIA DE SOFTWARE

A engenharia de software orientou a definição do que o sistema deve fazer e o planejamento do seu desenvolvimento. Este capítulo apresenta o levantamento e a especificação dos requisitos, o planejamento com o framework Scrum e a modelagem da solução em UML, além de delimitar o escopo que foi efetivamente prototipado neste trabalho.

4.1 Levantamento e especificação de requisitos

Os requisitos foram levantados a partir da caracterização da organização e dos problemas identificados na seção 2. Adotou-se a numeração RFxx para os requisitos funcionais e RNFxx para os requisitos não funcionais, de modo a permitir a rastreabilidade entre a especificação, a modelagem e a implementação. A distinção entre requisitos funcionais e não funcionais segue Sommerville (2018), para quem os requisitos não funcionais costumam ser tão críticos quanto os funcionais, por afetarem a arquitetura do sistema como um todo.

Os principais requisitos funcionais são: RF01 — autenticar usuários internos, negando o acesso após três tentativas inválidas; RF05 — cadastrar e manter produtos, categorias, filiais e usuários (administrador); RF06 — ativar e desativar produtos por filial (gerente); RF07 — exibir o cardápio ativo da filial ao cliente; RF09 — filtrar o cardápio por alérgeno; RF10 — expor o catálogo disponível às maquininhas dos garçons por meio de uma API (Application Programming Interface); RF11 — registrar em histórico toda alteração de disponibilidade; RF12 — gerar relatórios de indisponibilidade atual e de frequência de esgotamento; e RF13 — apresentar o cardápio com informação estruturada de alérgenos e em dois idiomas.

Os principais requisitos não funcionais são: RNF01 — a atualização de disponibilidade deve propagar-se a todos os canais em até 30 segundos; RNF02 — a interface pública deve atender à WCAG 2.2 nível AA e à ABNT NBR 17225; RNF03 — o cardápio deve estar disponível em português e inglês, com o idioma marcado no código da página; RNF04 — as senhas devem ser armazenadas como hash (bcrypt), nunca em texto puro, e o acesso das maquininhas deve usar token; RNF05 — a solução deve observar a Lei Geral de Proteção de Dados Pessoais (LGPD), sem cadastro de cliente nem cookies de rastreamento; e RNF06 — a arquitetura deve ser tolerante a falhas, com replicação de dados e link de contingência nas unidades.

4.2 Planejamento com o framework Scrum

O desenvolvimento foi planejado com o framework ágil Scrum, adequado a um escopo que evolui de forma incremental. Definiram-se os papéis de Product Owner, responsável por priorizar o backlog segundo o valor para o negócio, Scrum Master, responsável por remover impedimentos, e a equipe de desenvolvimento. O product backlog reuniu os requisitos funcionais e não funcionais em itens priorizados. Os papéis, eventos e artefatos adotados seguem o Guia do Scrum, que define o framework como uma estrutura leve para gerar valor por meio de soluções adaptativas para problemas complexos (Schwaber; Sutherland, 2020).

O trabalho foi organizado em sprints curtas, cada uma entregando um incremento verificável: a primeira sprint concentrou o levantamento de requisitos e a modelagem UML; a segunda, a modelagem do banco de dados; a terceira, a simulação em C da lógica de disponibilidade; a quarta, o projeto de rede e da arquitetura em nuvem; e a quinta, a avaliação de acessibilidade, inclusão e segurança. Ao final de cada sprint, o incremento foi revisado em relação aos critérios de aceitação derivados dos requisitos.

4.2.1 Backlog do produto

A partir dos requisitos, construiu-se o product backlog, que reúne os itens de valor priorizados para as sprints. O Quadro 2 apresenta os principais itens.

Quadro 2 – Product backlog (itens priorizados)

ID	**Item (história do usuário)**	**Prioridade**
B01	Como cliente, visualizar o cardápio ativo da filial (RF07).	Alta
B02	Como cliente, filtrar o cardápio por alérgeno (RF09/RF13).	Alta
B03	Como gerente, ativar/desativar produtos da minha filial (RF06).	Alta
B04	Como sistema, registrar em histórico cada alteração (RF11).	Alta
B05	Como administrador, manter produtos, categorias e filiais (RF05).	Média

ID	Item (história do usuário)	Prioridade
B06	Como administrador, consultar relatórios de indisponibilidade e frequência (RF12).	Média
B07	Como cliente, alternar o idioma entre português e inglês (RNF03).	Média
B08	Como maquininha, consultar o catálogo disponível via API (RF10).	Baixa

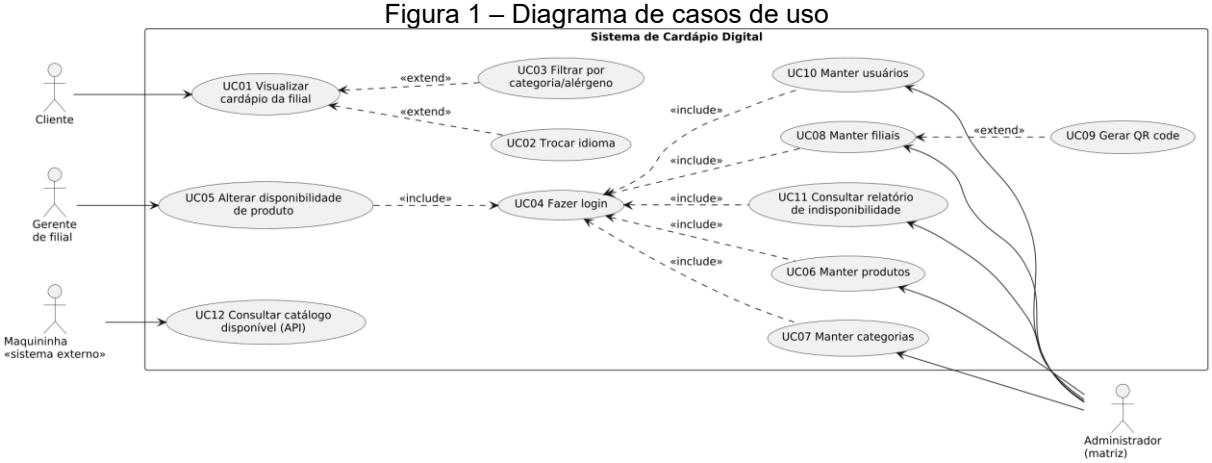
Fonte: elaborado pelos autores (2026).

4.3 Modelagem da solução em UML

4.3.1 Diagrama de casos de uso

O diagrama de casos de uso representa as interações entre os atores e o sistema. Os atores são o Cliente, que visualiza o cardápio, troca o idioma e filtra por categoria ou alérgeno; o Gerente de filial, que faz login e altera a disponibilidade dos produtos da sua unidade; o Administrador da matriz, que mantém produtos, categorias, filiais e usuários, gera o QR code e consulta os relatórios; e a Maquininha, sistema externo que consulta o catálogo disponível por API. Os casos de uso de manutenção e de alteração de disponibilidade incluem o caso Fazer login, e a geração de QR code estende a manutenção de filiais. O diagrama de casos de uso é apresentado na Figura 1.

Figura 1 – Diagrama de casos de uso



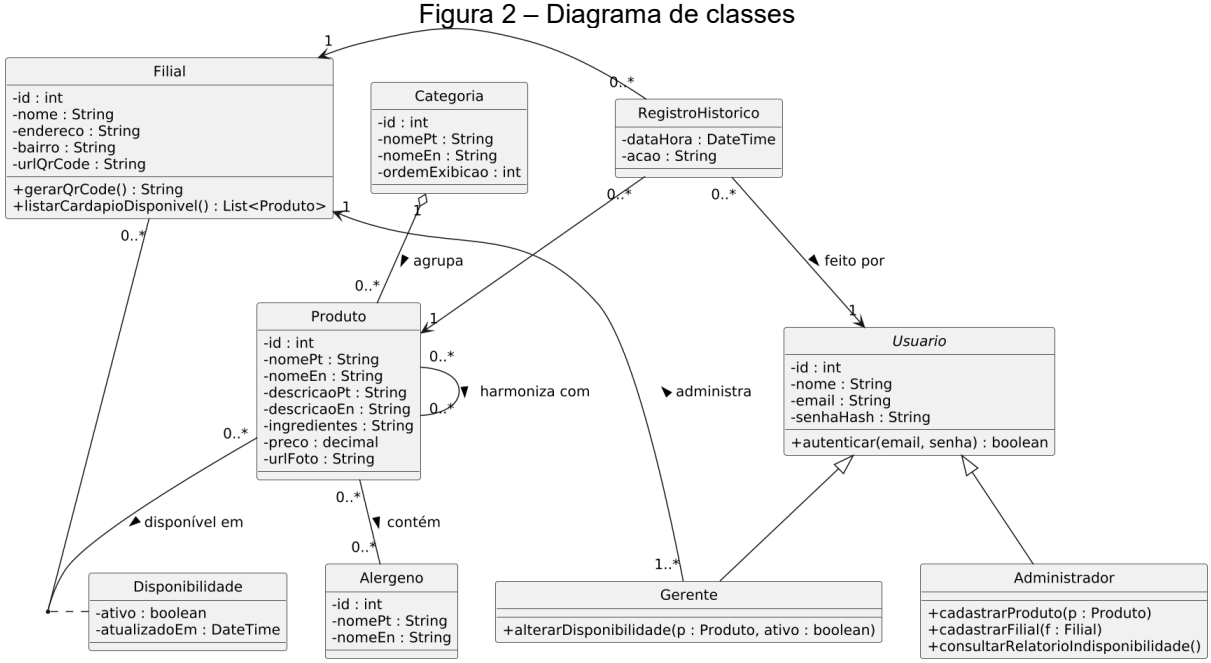
Fonte: elaborado pelos autores (2026).

4.3.2 Diagrama de classes

O diagrama de classes descreve a estrutura estática do domínio. As classes centrais são Filial, Categoria, Produto, Disponibilidade, Alérgeno, RegistroHistórico e Usuário, esta última especializada em Gerente e Administrador por herança. Um

Produto pertence a uma Categoria, contém zero ou mais Alérgenos e possui um registro de Disponibilidade por Filial; cada alteração de disponibilidade gera um RegistroHistórico associado ao Usuário que a realizou. O modelo prevê ainda a harmonização entre pratos e vinhos. O diagrama de classes é apresentado na Figura 2.

Figura 2 – Diagrama de classes

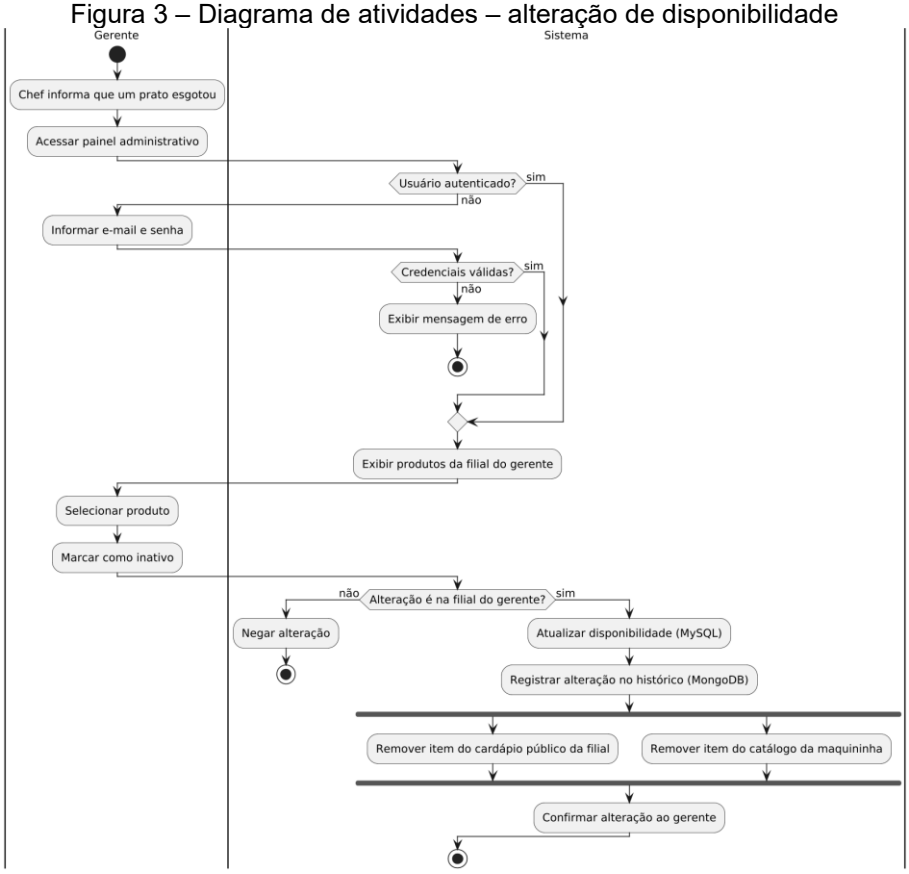


Fonte: elaborado pelos autores (2026).

4.3.3 Diagrama de atividades e fluxogramas

O diagrama de atividades detalha o processo de alteração de disponibilidade, desde o aviso do chef de que um prato esgotou até a confirmação ao gerente, passando pela autenticação, pela seleção do produto, pela verificação de que a alteração ocorre na filial do próprio gerente e pela gravação da nova disponibilidade e do registro de histórico no banco relacional (MySQL), em uma única transação, com posterior replicação do evento para o histórico em MongoDB. Dois fluxogramas complementam a modelagem: o da alteração de disponibilidade e o da exibição do cardápio da filial, ambos refletidos na simulação em C descrita no capítulo 6. O diagrama de atividades correspondente é apresentado na Figura 3.

Figura 3 – Diagrama de atividades – alteração de disponibilidade



Fonte: elaborado pelos autores (2026).

4.4 Escopo do produto e da simulação

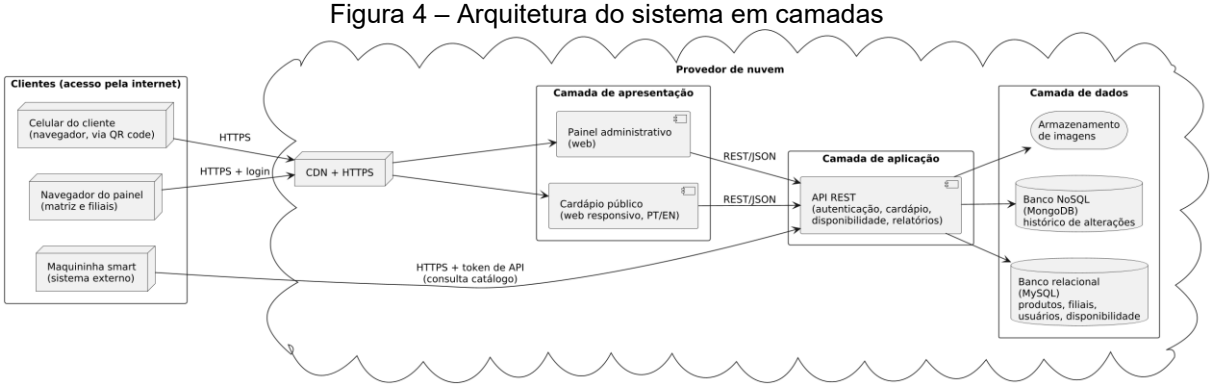
A modelagem contempla a solução completa de cardápio digital para a rede, incluindo os papéis e processos de atendimento. A prototipação deste trabalho, contudo, concentra-se no núcleo que resolve o problema principal: o controle de disponibilidade de produtos por filial, a sua visualização pelo cliente e o seu registro em histórico. Esse núcleo foi implementado na simulação em C (capítulo 6) e sustentado pelo modelo de dados (capítulo 5); os demais módulos ficam especificados e modelados para evolução futura, conforme discutido no capítulo 9.

4.5 Arquitetura do sistema

A arquitetura da solução organiza-se em três camadas, seguindo o padrão de arquitetura em camadas (Sommerville, 2018). A camada de apresentação reúne o cardápio público (web responsivo, acessível por QR code) e o painel administrativo; a camada de aplicação concentra a API REST (Representational State Transfer) responsável por autenticação, cardápio, disponibilidade e relatórios; e a camada de dados abrange o banco relacional (MySQL), o banco NoSQL do histórico (MongoDB) e o armazenamento das imagens. Os clientes acessam a solução pela internet e as

maquininhas consomem o catálogo por API. A Figura 4 apresenta essa visão em camadas.

Figura 4 – Arquitetura do sistema em camadas



Fonte: elaborado pelos autores (2026).

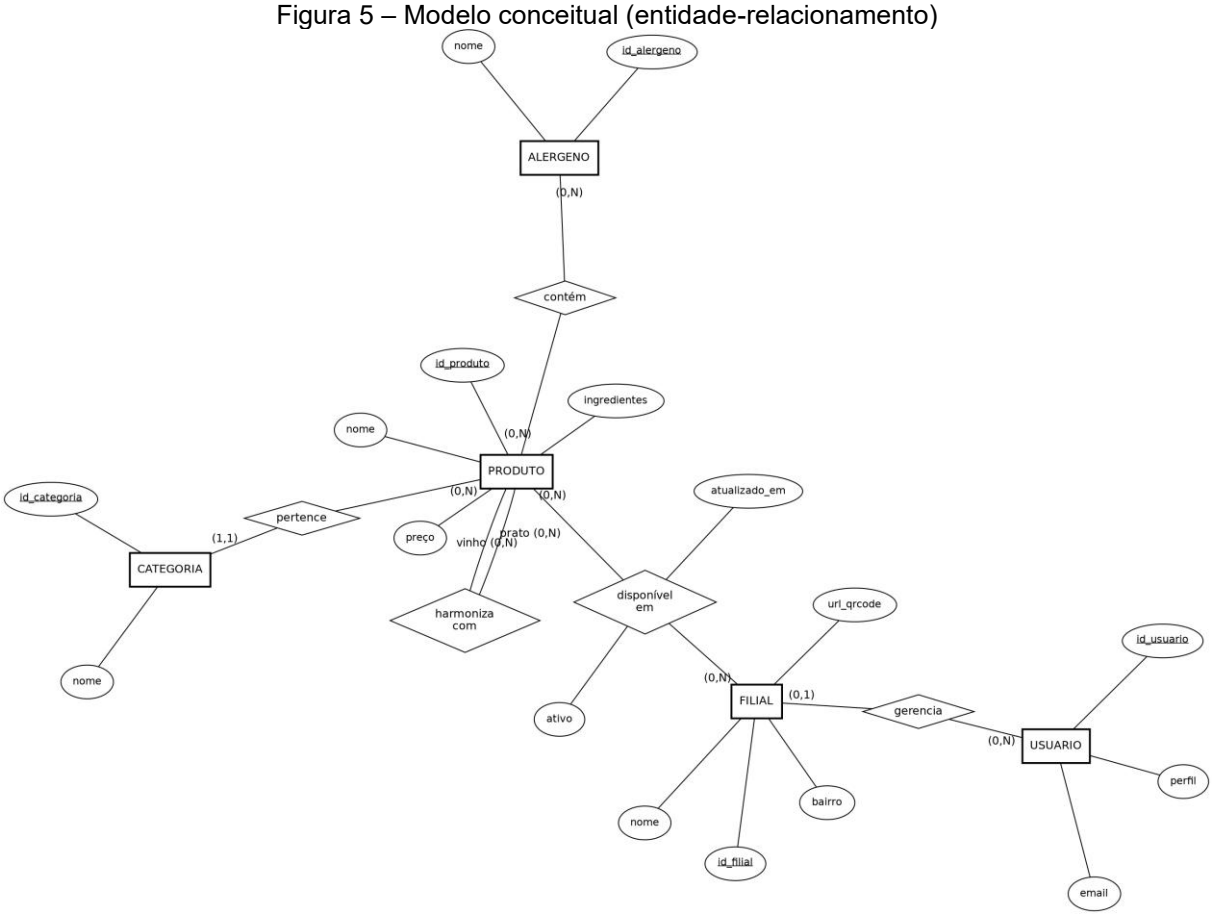
5 MODELAGEM DE BANCO DE DADOS

A modelagem de dados partiu dos requisitos e da modelagem UML e seguiu as etapas clássicas do projeto de banco de dados descritas por Elmasri e Navathe (2018): modelo conceitual, modelo lógico e modelo físico. O banco relacional foi implementado em MySQL e discute-se o uso de um banco NoSQL para o histórico de alterações.

5.1 Modelo conceitual

O modelo conceitual, em notação entidade-relacionamento, identifica as entidades do domínio e seus relacionamentos, sem preocupação com a tecnologia. As entidades são Filial, Usuário, Categoria, Produto e Alérgeno. Um Produto pertence a uma Categoria (1,1) e está disponível em zero ou mais Filiais; um Produto pode conter vários Alérgenos e um Alérgeno pode estar em vários Produtos (relacionamento N:N); uma Filial é gerenciada por um ou mais Usuários. Prevê-se ainda o auto-relacionamento de harmonização entre pratos e vinhos. A Figura 5 apresenta o modelo conceitual.

Figura 5 – Modelo conceitual (entidade-relacionamento)



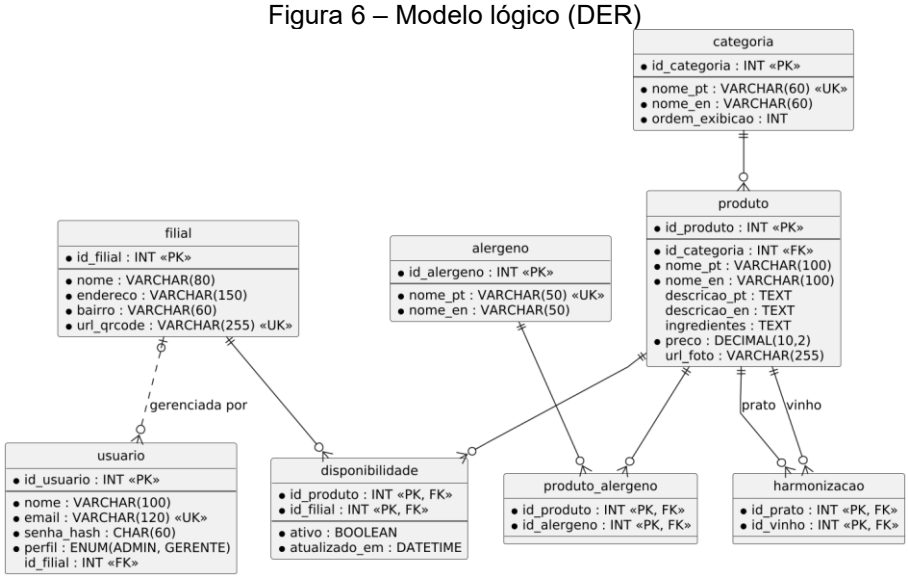
Fonte: elaborado pelos autores (2026).

5.2 Modelo lógico

No modelo lógico, os relacionamentos N:N foram resolvidos com tabelas associativas e as entidades foram normalizadas até a terceira forma normal. A disponibilidade de um produto em uma filial tornou-se a tabela associativa disponibilidade (id_produto, id_filial, ativo, atualizado_em); a relação entre produtos e alérgenos tornou-se produto_alergeno (id_produto, id_alergeno); e a harmonização entre prato e vinho tornou-se uma tabela com duas chaves estrangeiras para produto. Cada tabela recebeu chave primária e as chaves estrangeiras correspondentes.

A normalização foi verificada forma a forma, segundo as definições de Elmasri e Navathe (2018). Na primeira forma normal (1FN), eliminaram-se os atributos multivalorados: a lista de alérgenos de um prato, que no cardápio impresso aparecia como texto livre, passou a ser representada por linhas da tabela produto_alergeno, uma por par produto–alérgeno. Na segunda forma normal (2FN), garantiu-se que, nas tabelas de chave composta, todo atributo dependa da chave inteira: em disponibilidade, os campos ativo e atualizado_em dependem do par (id_produto, id_filial), enquanto nome e preço, que dependem apenas do produto, permanecem em produto. Na terceira forma normal (3FN), removeram-se as dependências transitivas: o nome da categoria e sua ordem de exibição ficam na tabela categoria, referenciada por id_categoria, em vez de se repetirem em cada produto. A Figura 6 apresenta o modelo lógico, representado no diagrama entidade-relacionamento (DER).

Figura 6 – Modelo lógico (DER)



Fonte: elaborado pelos autores (2026).

5.3 Modelo físico em MySQL

O modelo físico foi implementado no script modelo_fisico_mysql.sql (Apêndice B), com as tabelas filial, usuario, dispositivo, categoria, produto, alergeno, produto_alergeno, harmonizacao, disponibilidade e historico_disponibilidade. Os textos visíveis ao cliente foram duplicados em colunas _pt e _en (por exemplo nome_pt e nome_en) para atender ao requisito de bilinguismo. A tabela dispositivo guarda o token de acesso das maquininhas (RF10), armazenado como hash.

A integridade foi reforçada por restrições: uma restrição CHECK garante que o perfil ADMIN não tem filial e que o perfil GERENTE tem filial obrigatória; outra garante preço maior que zero; e, na harmonização, uma restrição impede que um prato harmonize consigo mesmo. Dois gatilhos (triggers) validam, na inserção e na atualização, que o item indicado como vinho pertence de fato à categoria de vinhos. Para acelerar as consultas mais frequentes do cardápio e dos relatórios, foram criados índices na tabela disponibilidade, sobre (id_filial, ativo), e na tabela de histórico, sobre (id_filial, data_hora).

5.4 Banco NoSQL para o histórico de alterações

O histórico de disponibilidade é uma série de eventos somente-anexação (append-only), de alto volume e leitura orientada a filtros por filial e por período. Para esse caso, discute-se o uso de um banco NoSQL orientado a documentos, como o MongoDB (Elmasri; Navathe, 2018), no qual cada alteração é um documento com produto, filial, usuário, ação e data/hora. Essa abordagem favorece a escrita sequencial, a escalabilidade horizontal e a flexibilidade de esquema, enquanto o banco relacional permanece como fonte de verdade do catálogo, da disponibilidade corrente e do próprio histórico; o MongoDB recebe uma cópia dos eventos, de forma assíncrona, para as consultas analíticas. A simulação em C representa esse histórico de forma simplificada no arquivo historico.txt.

A sincronização entre os dois bancos segue o padrão de caixa de saída transacional (transactional outbox). A própria tabela historico_disponibilidade faz o papel de caixa de saída: o evento é gravado na mesma transação que altera a disponibilidade (seção 5.5), garantindo que ambos sejam confirmados ou desfeitos juntos (Elmasri; Navathe, 2018). A API nunca escreve diretamente no MongoDB, o

que evita a escrita dupla (dual write), situação em que um banco poderia ser atualizado e o outro não.

A cópia para o MongoDB é feita por um serviço de replicação independente da API, que consulta o MySQL a cada poucos segundos e lê, em ordem crescente e em lotes, os registros cujo id_historico é maior que o último ponto de controle replicado. São considerados apenas registros com alguns segundos de idade, para não saltar eventos de transações concorrentes confirmadas fora da ordem de numeração. Cada evento é inserido no MongoDB com o próprio id_historico como identificador do documento (_id) e com confirmação de escrita pela maioria das réplicas; somente após essa confirmação o ponto de controle avança, sendo guardado em uma coleção de controle. Como a reinserção de um mesmo evento é rejeitada pela chave _id, a entrega ocorre ao menos uma vez, mas o efeito final é o de uma única gravação por evento, isto é, a operação é idempotente (Coulouris et al., 2013).

Os principais cenários de falha na replicação e a respectiva recuperação são:

a) MongoDB indisponível: o serviço de replicação não consegue gravar, o ponto de controle não avança e novas tentativas são feitas com intervalos crescentes (espera exponencial); os eventos permanecem guardados no MySQL e são enviados quando o MongoDB volta, sem perda;

b) falha do serviço de replicação após gravar no MongoDB e antes de atualizar o ponto de controle: ao reiniciar, o lote é reprocessado e os eventos já gravados são rejeitados pela chave _id, sem duplicidade;

c) transação revertida no MySQL: se a transação é desfeita (rollback), o registro de histórico também não é gravado e, portanto, nunca é replicado; as lacunas que isso deixa na numeração de id_historico não afetam a leitura por ponto de controle;

d) divergência ou perda de dados no MongoDB, por exemplo após a restauração de um backup: uma rotina diária de conciliação compara a quantidade de eventos por filial e por dia nos dois bancos e reenvia os que faltam; em último caso, o ponto de controle é zerado e o histórico é reconstruído integralmente a partir do MySQL, que é a fonte de verdade;

e) atraso na replicação: a defasagem é monitorada pela diferença entre o maior id_historico do MySQL e o ponto de controle, com alerta quando ultrapassa alguns minutos; esse atraso afeta apenas os relatórios analíticos,

e não o cardápio, pois o cardápio público e as maquininhas leem a disponibilidade corrente do MySQL, preservando o RNF01.

Desse modo, o projeto trabalha com dois níveis de consistência: a disponibilidade corrente e o seu registro de histórico são gravados de forma atômica no MySQL, e a disponibilidade é propagada aos canais em até 30 segundos (RNF01), enquanto a cópia analítica no MongoDB é eventualmente consistente, podendo ficar temporariamente atrasada, mas sempre convergindo para o conteúdo do banco relacional (Coulouris et al., 2013).

5.5 Integridade, transações e segurança dos dados

A alteração de disponibilidade e o seu registro no histórico constituem uma operação única do ponto de vista do negócio e, no banco relacional, são tratados em uma transação, de modo que ou ambos são gravados ou nenhum é. As chaves estrangeiras usam exclusão em cascata quando apropriado, preservando o histórico. Em conformidade com o RNF04, as senhas dos usuários são armazenadas como hash bcrypt (coluna de 60 caracteres) e nunca em texto puro, princípio reforçado no capítulo 8.

5.6 Principais comandos SQL

A seguir destacam-se comandos SQL (Structured Query Language) representativos do modelo físico: a criação de uma tabela com suas restrições, a transação que altera a disponibilidade e registra o histórico (RF06 e RF11) e a consulta do cardápio ativo de uma filial (RF07). Os trechos estão simplificados; o script completo, no Apêndice B, obtém a filial a partir do gerente autenticado.

```
-- Criacao de tabela (trecho do modelo fisico)
CREATE TABLE produto (
    id_produto   INT AUTO_INCREMENT PRIMARY KEY,
    id_categoria INT NOT NULL,
    nome_pt      VARCHAR(100) NOT NULL,
    nome_en      VARCHAR(100) NOT NULL,
    preco        DECIMAL(10,2) NOT NULL CHECK (preco > 0),
    FOREIGN KEY (id_categoria) REFERENCES categoria (id_categoria)
);
-- RF06 + RF11: alterar disponibilidade e registrar no historico (transacao)
START TRANSACTION;
UPDATE disponibilidade SET ativo = 0, atualizado_em = NOW()
 WHERE id_produto = 2 AND id_filial = 1;
INSERT INTO historico_disponibilidade (id_produto, id_filial, id_usuario, acao)
VALUES (2, 1, 2, 'DESATIVAR');
COMMIT;
-- RF07: cardapio ativo de uma filial
SELECT p.nome_pt, c.nome_pt AS categoria, p.preco
  FROM produto p
  JOIN categoria c       ON c.id_categoria = p.id_categoria
  JOIN disponibilidade d ON d.id_produto   = p.id_produto
```

```
WHERE d.id_filial = 1 AND d.ativo = 1
ORDER BY c.ordem_exibicao, p.nome_pt;
```

6 PROGRAMAÇÃO ESTRUTURADA

A lógica central do controle de disponibilidade foi implementada em linguagem C, na forma de uma simulação em console, cujo código-fonte, o arquivo simulacao_cardapio.c, consta do Apêndice A. A escolha da programação estruturada permite evidenciar, de maneira didática e sem dependência de banco de dados externo, os algoritmos que sustentam o requisito principal do projeto. O pseudocódigo correspondente está no Apêndice C. A linguagem C oferece os recursos de controle de fluxo, funções, vetores, estruturas e arquivos empregados na simulação (Kernighan; Ritchie, 1989).

6.1 Estruturas de dados

O programa usa um vetor de registros (struct Produto) com identificador, nome, categoria, preço e a lista de alérgenos de cada item, e um vetor de registros (struct Usuario) com login, senha (mantida em texto puro apenas para fins didáticos, conforme a seção 6.7), perfil e filial. A disponibilidade é representada por uma matriz de inteiros disponibilidade[produto][filial], em que 1 indica item ativo e 0 indica item inativo em determinada unidade. Essa matriz traduz diretamente o relacionamento N:N entre produto e filial modelado no banco de dados.

6.2 Organização em funções

O código foi decomposto em funções de responsabilidade única, agrupadas por finalidade: entrada validada (lerTexto, lerInteiro, lerPreco); persistência (salvarDados, carregarDados, registrarHistorico); manipulação de produtos e da disponibilidade (buscarProdutoPorId, adicionarProduto, cadastrarProduto, alterarDisponibilidade); exibição e relatórios (exibirCardapio, listarTodos, relatorioIndisponibilidade, relatorioFrequencia); e controle de sessão (autenticar, escolherFilial, menuAdmin, menuGerente, main). Essa separação facilita a leitura, o teste e a manutenção.

6.3 Funcionalidades implementadas

A autenticação (RF01) concede acesso ao administrador ou ao gerente e nega o acesso após três tentativas. A alteração de disponibilidade (RF06) ocorre sempre na filial vinculada ao login do gerente, com o estado novo informado de forma explícita (ativar ou desativar), para que repetir a operação não desfaça a alteração. A

exibição do cardápio (RF07) tem duas visões: a do cliente, que mostra apenas itens ativos, e a do gerente, que mostra o status de cada item. O histórico (RF11) grava data/hora, ação, produto, filial e usuário a cada alteração. Os relatórios (RF12) apresentam a indisponibilidade atual por filial e a frequência de esgotamento, obtida pela leitura do histórico. Como extensão, implementou-se o filtro por alérgeno (RF09): o cliente pode informar um alérgeno para ocultar os pratos que o contêm, e a lista de alérgenos de cada item é exibida junto ao cardápio (RF13).

6.4 Persistência em arquivos

Para dispensar um servidor de banco de dados na simulação, os dados são persistidos em arquivos texto. O arquivo cardapio.txt guarda, em cada linha, o identificador, a categoria, o nome, o preço, a lista de alérgenos e os três indicadores de disponibilidade do produto. O arquivo historico.txt registra, em cada linha, data/hora, ação, identificador do produto, filial e login, servindo de base para o relatório de frequência. Esses arquivos representam, de forma simplificada, o papel do banco relacional e do histórico NoSQL descritos no capítulo 5.

6.5 Validação de entradas e robustez

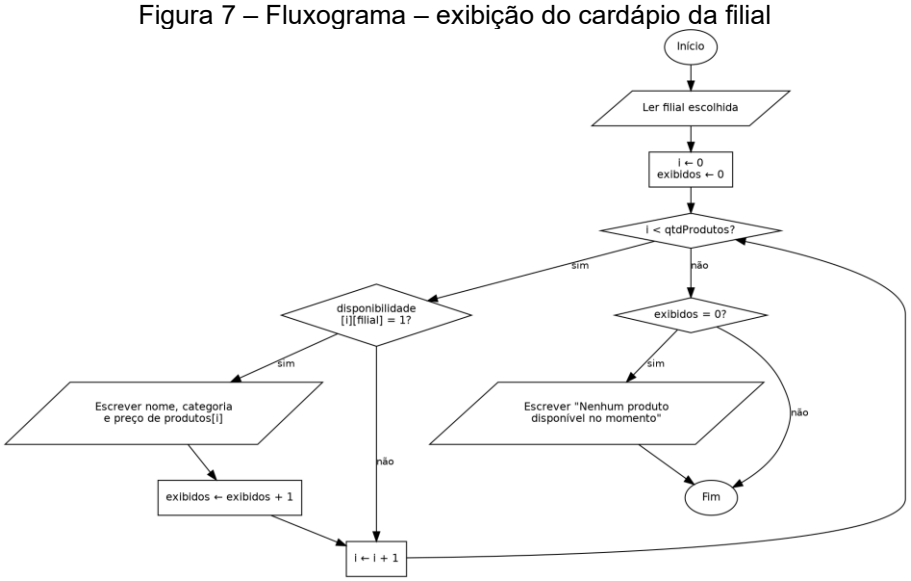
A leitura de dados é feita por funções que repetem a solicitação até receber um valor válido: números inteiros e preços são validados, aceitando-se vírgula ou ponto como separador decimal, e o caractere ';' é rejeitado no cadastro para não corromper o formato dos arquivos. A leitura de texto limita o tamanho do buffer e descarta o excedente da linha, evitando estouro de buffer. Essas validações tornam a simulação estável mesmo diante de entradas inesperadas do usuário.

A simulação emprega os principais recursos da programação estruturada: estruturas condicionais (if/else e switch nos menus e na verificação de disponibilidade), estruturas de repetição (laços for na varredura dos produtos e do-while nos menus), funções com passagem de parâmetros, vetores e matriz, e manipulação de arquivos-texto.

6.6 Fluxogramas

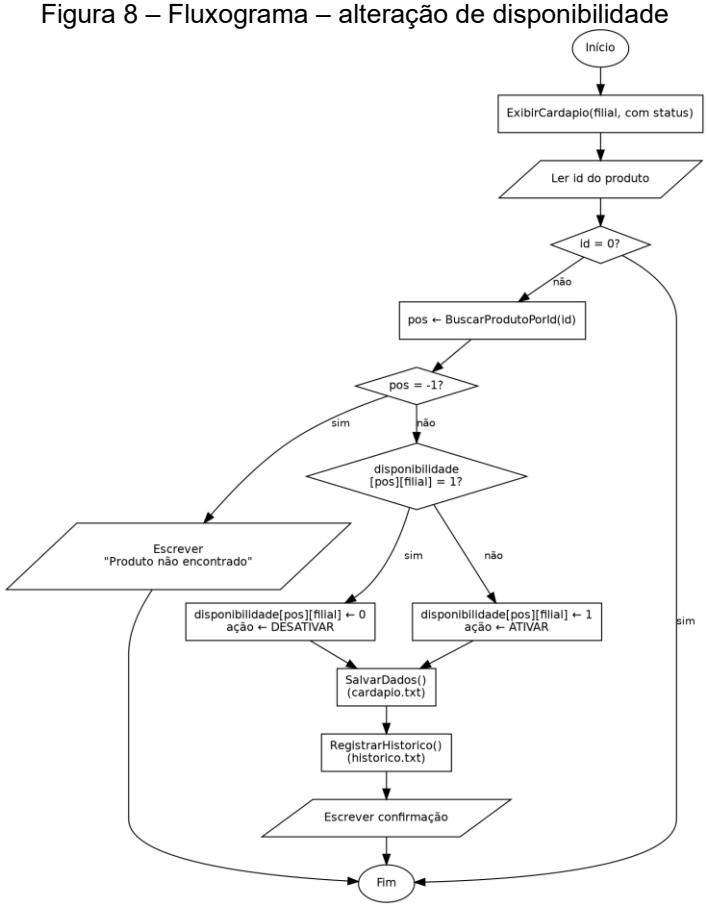
Os fluxogramas descrevem a lógica das duas operações centrais da simulação. As Figuras 7 e 8 apresentam, respectivamente, a exibição do cardápio da filial e a alteração de disponibilidade de um produto.

Figura 7 – Fluxograma – exibição do cardápio da filial



Fonte: elaborado pelos autores (2026).

Figura 8 – Fluxograma – alteração de disponibilidade



Fonte: elaborado pelos autores (2026).

6.7 Segurança: simulação acadêmica e implementação prevista

Para que a simulação pudesse ser executada sem servidor de banco de dados, algumas simplificações de segurança foram adotadas de forma deliberada e não devem ser tomadas como prática recomendada: os usuários de teste estão fixos no código-fonte, as senhas são guardadas e comparadas em texto puro (função

strcmp), o limite de três tentativas vale apenas por execução do programa e os arquivos cardapio.txt e historico.txt não têm controle de acesso. O próprio programa registra essa condição no cabeçalho do código e exibe um aviso de simulação acadêmica na tela de login.

Na implementação prevista para produção, cada simplificação tem um correspondente seguro. Nenhuma credencial fica no código ou no repositório: os usuários são cadastrados na tabela usuario do MySQL, e os segredos de conexão ficam em variáveis de ambiente ou em um cofre de segredos. A senha é armazenada somente como hash bcrypt com salt (coluna senha_hash, RNF04) e conferida pela função de verificação da biblioteca, em vez de uma comparação direta. As tentativas de login são limitadas por conta e por endereço IP, com bloqueio temporário e registro das falhas. O acesso ocorre apenas por HTTPS/TLS e, entre as unidades, por VPN; após o login, a sessão é mantida por um token com expiração, e a filial do gerente é verificada no servidor a cada requisição. Por fim, os arquivos texto dão lugar ao banco de dados acessado por um usuário de privilégio mínimo, com o histórico mantido em modo somente-anexação.

7 REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS

A solução é um sistema distribuído: as três unidades da rede e os clientes acessam uma aplicação hospedada em nuvem. Este capítulo descreve a arquitetura distribuída, a interligação das unidades (Wide Area Network – WAN) e a rede local típica de uma filial (Local Area Network – LAN), além das propriedades de sistemas distribuídos que sustentam o requisito de atualização rápida da disponibilidade. Segundo Coulouris et al. (2013), um sistema distribuído é aquele cujos componentes, localizados em computadores interligados em rede, comunicam-se e coordenam suas ações apenas por troca de mensagens.

7.1 Arquitetura distribuída em nuvem

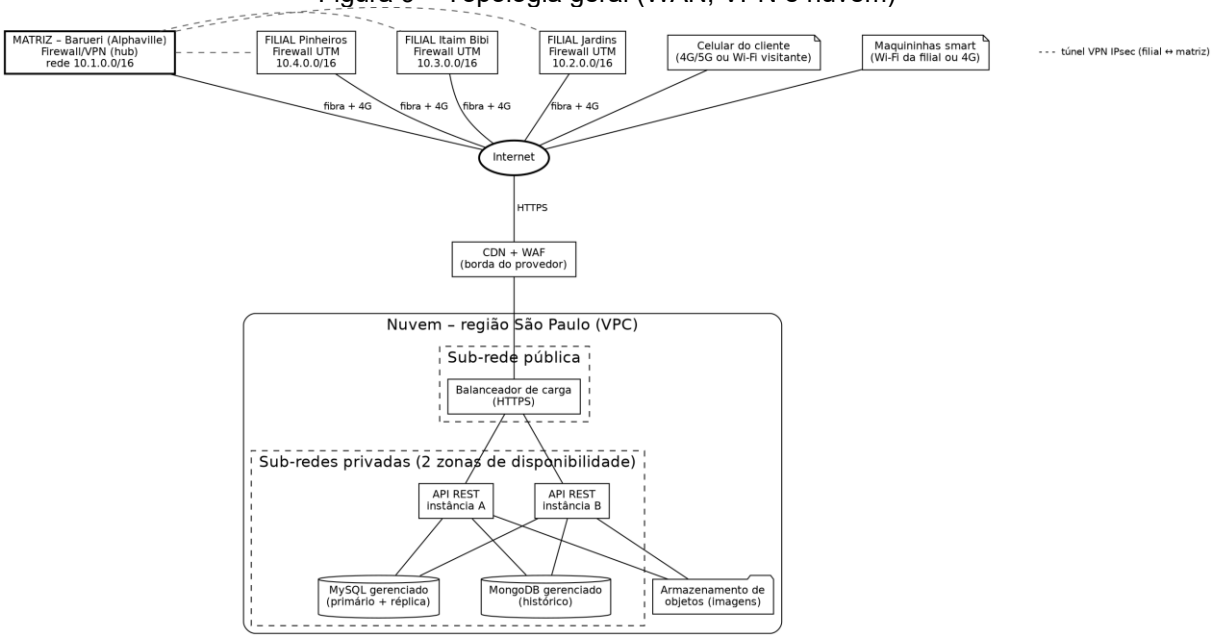
A aplicação foi projetada para uma nuvem pública na região de São Paulo, dentro de uma rede virtual privada (VPC). Na borda do provedor, uma rede de distribuição de conteúdo (Content Delivery Network – CDN) com firewall de aplicação web (Web Application Firewall – WAF) distribui o conteúdo estático e protege a aplicação; o tráfego chega por HTTPS (Hypertext Transfer Protocol Secure) a um balanceador de carga, que distribui as requisições entre instâncias da API REST (responsáveis por autenticação, cardápio, disponibilidade e relatórios). A persistência usa um MySQL gerenciado com réplica de leitura, um MongoDB gerenciado para o histórico e um serviço de armazenamento de objetos para as imagens dos pratos. As instâncias de aplicação e os bancos ficam em sub-redes privadas, distribuídas em duas zonas de disponibilidade.

A comunicação entre os serviços segue padrões abertos. O cardápio público consulta a API REST de forma anônima, trocando mensagens JSON (JavaScript Object Notation) sobre HTTPS; o painel do gerente usa o mesmo canal, autenticado por token de sessão; e as maquininhas, identificadas pelo token do dispositivo, mantêm uma conexão WebSocket com a API e recebem por push cada mudança de disponibilidade, com consulta periódica como alternativa em caso de queda, o que sustenta o prazo do RNF01. Além do registro transacional no MySQL, cada alteração é publicada em uma fila de mensagens, e um consumidor a replica no MongoDB do histórico, sem atrasar a resposta ao gerente.

7.2 Topologia WAN e interligação das unidades

A matriz, em Barueri, funciona como concentrador (hub) da rede corporativa (10.1.0.0/16) e interliga-se às filiais dos Jardins, de Itaim Bibi e de Pinheiros por túneis de rede privada virtual (Virtual Private Network – VPN) protegidos pelo IPsec, conjunto de protocolos de segurança do Internet Protocol (IP), conforme Kurose e Ross (2021), formando uma topologia em estrela. Cada unidade possui um firewall UTM (Unified Threat Management) e acesso à internet por fibra óptica, com link 4G/5G de contingência. O acesso dos clientes e das maquininhas à aplicação ocorre pela internet, enquanto o tráfego administrativo entre unidades e matriz trafega cifrado pelos túneis VPN. A Figura 9 apresenta a topologia geral.

Figura 9 – Topologia geral (WAN, VPN e nuvem)



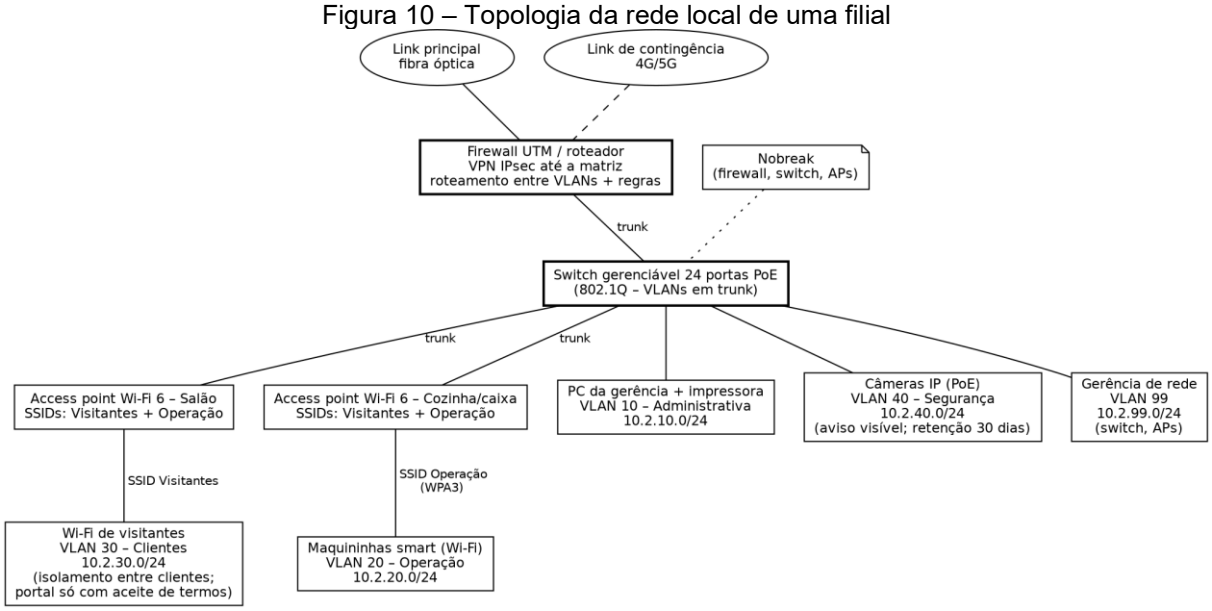
Fonte: elaborado pelos autores (2026).

7.3 Rede local de uma filial

A rede local de cada filial adota topologia em estrela a partir de um switch gerenciável de 24 portas com alimentação pela rede (Power over Ethernet – PoE) e suporte a 802.1Q. O firewall/roteador faz o roteamento entre as redes locais virtuais (Virtual LANs – VLANs), aplica as regras de segurança e estabelece o túnel VPN com a matriz; um nobreak mantém firewall, switch e pontos de acesso em funcionamento. Dois pontos de acesso Wi-Fi 6 cobrem o salão e a cozinha/caixa. O tráfego é segmentado em VLANs: administrativa (VLAN 10), operação e maquininhas (VLAN 20), visitantes/clientes (VLAN 30, isolada e com portal de aceite de termos), câmeras IP (VLAN 40) e gerência de rede (VLAN 99). A segmentação limita o alcance de

eventuais incidentes e separa o tráfego dos clientes do tráfego interno. Definidas pelo padrão IEEE (Institute of Electrical and Electronics Engineers) 802.1Q, as VLANs criam redes lógicas independentes sobre a mesma infraestrutura física (Kurose; Ross, 2021). A Figura 10 apresenta a topologia da rede local da filial.

Figura 10 – Topologia da rede local de uma filial



Fonte: elaborado pelos autores (2026).

7.4 Propriedades de sistemas distribuídos

O projeto busca disponibilidade e tolerância a falhas por meio da replicação do banco de dados, da distribuição das instâncias em duas zonas e do link de contingência nas unidades, já que a replicação é o principal meio de elevar a disponibilidade e a tolerância a falhas (Coulouris et al., 2013). A atualização da disponibilidade segue um modelo de consistência eventual: quando o gerente altera o estado de um produto, a mudança é gravada na nuvem e propagada ao cardápio público e às maquininhas em até 30 segundos (RNF01), atendendo ao diferencial de disponibilidade em tempo quase real. Já o histórico analítico é replicado de forma assíncrona e idempotente do MySQL para o MongoDB, com recuperação automática de falhas, conforme descrito na seção 5.4. A segurança do sistema distribuído apoia-se na cifragem do tráfego em trânsito pelo protocolo TLS (Transport Layer Security), base do HTTPS, nos túneis VPN entre as unidades e na segmentação por VLANs.

7.5 Endereçamento IP e escalabilidade

O endereçamento segue um plano por unidade e por função. A matriz usa a faixa 10.1.0.0/16 e cada filial uma faixa própria (por exemplo, 10.2.0.0/16 nos Jardins), segmentada em sub-redes por VLAN, conforme o Quadro 3.

Quadro 3 – Plano de VLANs e sub-redes de uma filial (Jardins)

VLAN	**Finalidade**	**Sub-rede**
10	Administrativa (PCs e impressora)	10.2.10.0/24
20	Operação e maquininhas	10.2.20.0/24
30	Wi-Fi de visitantes (clientes)	10.2.30.0/24
40	Câmeras IP	10.2.40.0/24
99	Gerência de rede	10.2.99.0/24

Fonte: elaborado pelos autores (2026).

A escalabilidade é obtida na nuvem pelo balanceador de carga, que distribui as requisições entre múltiplas instâncias da API REST, e pela possibilidade de ampliar esse número de forma elástica conforme a demanda. O banco relacional usa réplica de leitura para aliviar as consultas do cardápio, e a distribuição das instâncias e dos bancos em duas zonas de disponibilidade soma escalabilidade e tolerância a falhas.

8 DIREITOS HUMANOS, INCLUSÃO E SEGURANÇA

A solução foi avaliada quanto à acessibilidade, à inclusão digital, à informação ao consumidor, à proteção de dados e aos direitos humanos. Este capítulo demonstra como o projeto atende à legislação e às normas aplicáveis e como esses valores foram incorporados desde a concepção.

8.1 Acessibilidade digital

O cardápio público foi especificado em HTML estruturado, e não como um PDF do cardápio impresso, de modo a ser compatível com leitores de tela e navegação por teclado. A solução segue as diretrizes WCAG 2.2 no nível AA e a ABNT NBR 17225, observando os princípios de perceptibilidade, operabilidade, compreensibilidade e robustez: contraste adequado, textos alternativos nas imagens dos pratos, marcação semântica e foco visível. A Lei Brasileira de Inclusão (Lei 13.146/2015), no seu artigo 63, torna obrigatória a acessibilidade dos sítios da internet de empresas com representação comercial no Brasil (Brasil, 2015), o que fundamenta essa exigência.

8.2 Inclusão digital

A pesquisa TIC Domicílios indica que uma parcela significativa da população ainda não tem acesso à internet, com destaque para as pessoas idosas (Comitê Gestor da Internet no Brasil, 2025). Para não excluir esse público, o projeto posiciona o cardápio digital como complemento, e não substituto, do atendimento: o cardápio impresso e o garçom continuam disponíveis, e o acesso ao cardápio digital não exige instalação de aplicativo nem cadastro do cliente.

8.3 Informação ao consumidor e alérgenos

O Código de Defesa do Consumidor (Lei 8.078/1990) assegura o direito à informação adequada e clara sobre os produtos (Brasil, 1990), e a Anvisa exige a declaração dos principais alimentos alergênicos na rotulagem (Agência Nacional de Vigilância Sanitária, 2022). Alinhado a isso e ao Consenso Brasileiro sobre Alergia Alimentar, o projeto estrutura a informação de alérgenos no banco de dados e permite filtrar o cardápio por alérgeno, exibindo um aviso de que a informação é de referência e de que, em caso de alergia grave, o cliente deve confirmar com o atendimento. Esse recurso foi também demonstrado na simulação em C (RF09 e RF13).

8.4 Proteção de dados e privacidade (LGPD)

A solução adota a privacidade desde a concepção: o cliente consulta o cardápio sem se cadastrar e sem cookies de rastreamento, de modo que praticamente não há coleta de dados pessoais do consumidor, em linha com os princípios de minimização e finalidade da LGPD (Brasil, 2018). Os dados de acesso interno são protegidos com hash bcrypt para as senhas (RNF04) e token para as maquininhas; o tráfego é cifrado por HTTPS e VPN; e as câmeras das unidades, segmentadas em VLAN própria, têm retenção limitada e aviso visível, respeitando a base legal e a transparência.

8.5 Direitos humanos e responsabilidade social

Ao garantir que pessoas com deficiência visual possam consultar o cardápio de forma autônoma, que pessoas com restrições alimentares recebam informação confiável e que o público sem acesso digital continue bem atendido, a solução concretiza princípios de dignidade, não discriminação e igualdade de acesso. A oferta bilíngue (português e inglês) amplia o alcance a clientes estrangeiros, reforçando o caráter inclusivo da proposta.

8.6 Aspectos éticos e impactos sociais

Do ponto de vista social, a solução foi concebida para apoiar, e não substituir, o trabalho das equipes: o cardápio digital reduz os pedidos de itens esgotados e libera o garçom para o atendimento, sem eliminar sua função. A implantação prevê o treinamento de gerentes e garçons, para que a mudança de processo não exclua os funcionários menos familiarizados com tecnologia, e o risco de exclusão de clientes sem smartphone ou sem conexão é mitigado pela manutenção dos canais tradicionais (seção 8.2).

No plano ético, o histórico registra o usuário responsável por cada alteração, o que é necessário à auditoria, mas poderia ser desvirtuado como instrumento de vigilância individual. Define-se, por isso, que o histórico serve à gestão da operação, e não à punição de funcionários, com acesso restrito ao administrador e ao gerente da própria filial; o mesmo vale para as câmeras, usadas apenas para a finalidade informada. Essas diretrizes alinham-se à Declaração Universal dos Direitos Humanos, que assegura a igualdade sem distinção (art. 2º) e a proteção contra interferências arbitrárias na vida privada (art. 12) (Organização das Nações Unidas, 1948).

9 INTEGRAÇÃO DA SOLUÇÃO

A integração da solução articula os resultados das disciplinas em um único fluxo coerente, do problema de negócio até a entrega técnica. Este capítulo descreve a visão integrada do sistema e como cada componente se conecta aos demais.

9.1 Visão integrada do funcionamento

O cliente acessa, por QR code, o cardápio público da filial; a página, em HTML acessível, consulta a API REST, que retorna apenas os produtos ativos naquela unidade, com seus alérgenos e no idioma escolhido. Quando um prato se esgota, o gerente faz login no painel e altera a disponibilidade da sua filial; a API grava a mudança no banco relacional e registra o evento no histórico, propagando o novo estado ao cardápio do cliente e às maquininhas dos garçons em até 30 segundos. Assim, o que o cliente vê, o que o garçom oferece e o que a matriz acompanha permanecem sincronizados.

9.2 Integração entre as disciplinas

A engenharia de software definiu os requisitos e a modelagem que orientaram todo o projeto; a modelagem de banco de dados transformou esse domínio em um esquema íntegro e consultável; a programação estruturada validou, em C, a lógica central de disponibilidade que a API exporia em produção; as redes e os sistemas distribuídos proveem a infraestrutura que entrega essa lógica de forma segura e disponível às unidades e aos clientes; e a análise de direitos humanos, inclusão e segurança assegura que a entrega seja acessível, inclusiva e conforme a LGPD. Cada disciplina resolve uma camada do mesmo problema.

9.3 Da simulação ao sistema em produção

A simulação em C representa, de forma concentrada, as regras que no sistema real residiriam na camada de aplicação: autenticação, controle de disponibilidade por filial, exibição filtrada e histórico. A evolução para produção substituiria os arquivos texto pelos bancos MySQL e MongoDB, exporia as funções como endpoints da API REST, acrescentaria o front-end HTML acessível e a integração com as maquininhas e implantaria o conjunto na arquitetura em nuvem descrita no capítulo 7, preservando os requisitos não funcionais de desempenho,

segurança e acessibilidade. As simplificações de segurança adotadas na simulação e o seu equivalente em produção foram detalhados na seção 6.7.

9.4 Benefícios esperados e limitações

Entre os benefícios esperados para a organização estão a redução de pedidos frustrados por itens esgotados, a melhor experiência e a maior autonomia do cliente, a padronização da informação entre unidades e a visão consolidada da matriz sobre o que esgota em cada filial. Como limitações, a validação restringiu-se à simulação em console da lógica central, com dados fictícios, sem a API, o front-end e as maquininhas em funcionamento — pontos discutidos na seção 9.3 e na conclusão.

10 CONCLUSÃO

O trabalho atingiu o objetivo proposto ao apresentar, de forma integrada, o projeto de um cardápio digital acessível com controle de disponibilidade por filial para a rede Sabor & Cia Bistrô. Os objetivos específicos foram cumpridos: os requisitos foram levantados e a solução foi modelada em UML; o banco de dados relacional foi modelado e implementado, com discussão do uso de NoSQL para o histórico; a lógica central foi validada por uma simulação em C; a infraestrutura de rede e a arquitetura em nuvem foram projetadas; e a solução foi avaliada quanto à acessibilidade, à inclusão digital, à proteção de dados e aos direitos do consumidor. Quanto ao problema de pesquisa, o trabalho mostra que uma rede com várias unidades pode manter o cardápio sincronizado com a disponibilidade real centralizando o catálogo na matriz, delegando a cada gerente o estado dos pratos da sua filial e distribuindo essa informação, a partir de uma única fonte, ao cardápio acessível e bilíngue do cliente e às maquininhas dos garçons.

A principal contribuição é demonstrar que a defasagem de informação entre o cardápio e a disponibilidade real pode ser resolvida com uma solução de baixo custo de adoção para o cliente, acessível e aderente à legislação brasileira, sem transformar o recurso digital em barreira de acesso. Os diferenciais — atualização em até 30 segundos, cardápio em HTML acessível e bilíngue, informação estruturada de alérgenos e privacidade desde a concepção — distinguem a proposta das soluções correntes de mercado baseadas em PDF.

Como limitações, registra-se que a organização é fictícia, que os dados utilizados são de exemplo e que a implementação se restringiu a uma simulação em console da lógica central, sem a API REST, o front-end e a integração com as maquininhas em funcionamento. Esses pontos constituem os trabalhos futuros: implementar a API e o front-end acessível, integrar as maquininhas, substituir os arquivos texto pelos bancos MySQL e MongoDB, adicionar testes automatizados e implantar a solução na arquitetura em nuvem descrita, avaliando o desempenho e a conformidade em um ambiente real.

REFERÊNCIAS

AGÊNCIA NACIONAL DE VIGILÂNCIA SANITÁRIA. **Resolução da Diretoria Colegiada – RDC nº 727, de 1º de julho de 2022**. Dispõe sobre a rotulagem dos alimentos embalados. Brasília, DF: Anvisa, 2022.

ASSOCIAÇÃO BRASILEIRA DE BARES E RESTAURANTES. **Cardápio por QR Code é uma questão de mercado, não de lei, diz Abrasel**. [S. l.]: Abrasel, [2023]. Disponível em: https://abrasel.com.br/noticias/noticias/cardapio-por-qr-code-e-uma-questao-de-mercado-nao-de-lei-diz-abrasel/. Acesso em: 28 out. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 17225**: acessibilidade em conteúdo e aplicações web: requisitos. Rio de Janeiro: ABNT, 2025.

BRASIL. **Lei nº 8.078, de 11 de setembro de 1990**. Dispõe sobre a proteção do consumidor e dá outras providências. Brasília, DF: Presidência da República, 1990. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/l8078compilado.htm. Acesso em: 28 out. 2026.

BRASIL. **Lei nº 13.146, de 6 de julho de 2015**. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência). Brasília, DF: Presidência da República, 2015. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/l13146.htm. Acesso em: 28 out. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 12 nov. 2026.

COMITÊ GESTOR DA INTERNET NO BRASIL. **Pesquisa sobre o uso das tecnologias de informação e comunicação nos domicílios brasileiros**: TIC Domicílios 2025: principais resultados. São Paulo: CGI.br, 2025. Disponível em: https://cetic.br/media/analises/tic_domicilios_2025_principais_resultados.pdf. Acesso em: 28 out. 2026.

COULOURIS, George et al. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 7. ed. Barueri, SP: Atlas, 2022.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Censo 2022**: Brasil tem 14,4 milhões de pessoas com deficiência. Rio de Janeiro: IBGE, 2025. Disponível em: https://agenciadenoticias.ibge.gov.br/agencia-noticias/2012-agencia-de-noticias/noticias/43463-censo-2022-brasil-tem-14-4-milhoes-de-pessoas-com-deficiencia. Acesso em: 28 out. 2026.

ISKENDER, Ali; SIRAKAYA-TURK, Ercan; CARDENAS, David. Restaurant menus and COVID-19: implications for technology adoption in the post-pandemic era. **Consumer Behavior in Tourism and Hospitality**, v. 18, n. 4, p. 587-605, 2023. DOI: https://doi.org/10.1108/CBTH-11-2022-0194.

ISKENDER, Ali et al. Restaurant patrons' intentions toward QR code menus in the U.S. during COVID-19: acceptance of technology adoption model (ATAM). **Journal of Foodservice Business Research**, v. 27, n. 5, p. 497-522, 2024. DOI: https://doi.org/10.1080/15378020.2022.2133518.

KERNIGHAN, Brian W.; RITCHIE, Dennis M. **C**: a linguagem de programação padrão ANSI. Rio de Janeiro: Campus, 1989.

KOAY, Kian Yeik; ANG, Kersoon. Understanding consumers' intentions to use QR code menus in the post-COVID-19 pandemic. **British Food Journal**, v. 126, n. 5, p. 2172-2186, 2024. DOI: https://doi.org/10.1108/BFJ-10-2023-0880.

KUROSE, James F.; ROSS, Keith W. **Redes de computadores e a internet**: uma abordagem top-down. 8. ed. Porto Alegre: Bookman, 2021.

MARCONI, Marina de Andrade; LAKATOS, Eva Maria. **Fundamentos de metodologia científica**. 9. ed. Barueri, SP: Atlas, 2021.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS. **Declaração Universal dos Direitos Humanos**. Paris: ONU, 1948. Disponível em: https://www.unicef.org/brazil/declaracao-universal-dos-direitos-humanos. Acesso em: 12 nov. 2026.

RADKE, Taylor J. et al. Food allergy knowledge and attitudes of restaurant managers and staff: an EHS-Net study. **Journal of Food Protection**, v. 79, n. 9, p. 1588-1598, 2016. DOI: https://doi.org/10.4315/0362-028X.JFP-16-085.

SCHWABER, Ken; SUTHERLAND, Jeff. **O guia do Scrum**: o guia definitivo para o Scrum: as regras do jogo. [S. l.]: Scrum.org, 2020. Disponível em: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-PortugueseBR-3.0.pdf. Acesso em: 12 nov. 2026.

SOLÉ, Dirceu *et al.* Consenso Brasileiro sobre Alergia Alimentar: 2018 – Parte 1 – Etiopatogenia, clínica e diagnóstico. **Arquivos de Asma, Alergia e Imunologia**, v. 2, n. 1, p. 7-38, 2018.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.2**. [S. l.]: W3C, 2023. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 12 nov. 2026.

APÊNDICE A – CÓDIGO-FONTE DA SIMULAÇÃO EM C

```
/*
 * PIM II - Sabor & Cia Bistro
 * Simulador em console do Cardapio Digital
 *
 * Funcionalidades:
 * - Cliente: visualizar o cardapio ativo de uma filial
 * - Gerente: ativar/desativar produtos somente da propria filial
 * - Administrador: cadastrar produtos, listar todos e ver relatorios
 *    (indisponibilidade atual e frequencia de esgotamento)
 *
 * Estruturas usadas: vetor de structs (produtos e usuarios),
 * matriz (disponibilidade produto x filial) e arquivos texto
 * (cardapio.txt para os dados, historico.txt para o log de alteracoes).
 *
 * Obs.: os textos do programa estao sem acento para evitar problemas
 * de exibicao no console do Windows.
 *
 * AVISO DE SEGURANCA - SIMULACAO ACADEMICA
 * Por simplificacao didatica, os usuarios de teste estao fixos no codigo,
 * as senhas ficam em texto puro, os arquivos .txt nao tem controle de
 * acesso e o limite de 3 tentativas vale apenas por execucao. Isso NAO
 * representa o sistema real: em producao as senhas ficam somente como
 * hash bcrypt no banco (coluna usuario.senha_hash, RNF04), conforme a
 * secao 6.7 do relatorio e o script database/modelo_fisico_mysql.sql.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <ctype.h>

#define MAX_PRODUTOS  100
#define NUM_FILIAIS   3
#define NUM_USUARIOS  4
#define TAM_NOME      60
#define TAM_CATEGORIA 30
#define TAM_ALERGENOS 80
#define ARQ_DADOS     "cardapio.txt"
#define ARQ_HISTORICO "historico.txt"

#define PERFIL_ADMIN   1
#define PERFIL_GERENTE 2

/* ---------- Estruturas ---------- */

typedef struct {
    int id;
    char nome[TAM_NOME];
    char categoria[TAM_CATEGORIA];
    int preco_cent; /* preco em centavos; evita erro de arredondamento do float */
    char alergenos[TAM_ALERGENOS]; /* lista separada por virgula; "nenhum" se vazio */
} Produto;

typedef struct {
    char login[20];
    char senha[20];
    int perfil; /* PERFIL_ADMIN ou PERFIL_GERENTE */
    int filial; /* indice da filial do gerente; -1 para admin */
} Usuario;

/* ---------- Dados globais ---------- */

const char *FILIAIS[NUM_FILIAIS] = {"Jardins", "Itaim Bibi", "Pinheiros"};

Produto produtos[MAX_PRODUTOS]; /* vetor de produtos */
```

```
int disponibilidade[MAX_PRODUTOS][NUM_FILIAIS]; /* matriz: 1 = ativo, 0 = inativo */
int qtdProdutos = 0;

/* Credenciais de TESTE, fixas no codigo e em texto puro, APENAS para a
   simulacao. No sistema real os usuarios ficam na tabela usuario do banco,
   a senha e guardada so como hash bcrypt e conferida pela funcao de
   verificacao da biblioteca (RNF04); nenhuma senha fica no codigo-fonte. */
Usuario usuarios[NUM_USUARIOS] = {{"admin", "admin123", PERFIL_ADMIN, -1},
                                  {"jardins", "1234", PERFIL_GERENTE, 0},
                                  {"itaim", "1234", PERFIL_GERENTE, 1},
                                  {"pinheiros", "1234", PERFIL_GERENTE, 2}};

/* ---------- Funcoes de entrada ---------- */

/* Le uma linha de texto e remove o '\n' do final */
void lerTexto(const char *mensagem, char *destino, int tamanho) {
    printf("%s", mensagem);
    if (fgets(destino, tamanho, stdin) == NULL) { /* fim da entrada */
        printf("\nEntrada encerrada.\n");
        exit(0);
    }
    if (strchr(destino, '\n') == NULL) { /* linha maior que o buffer */
        int c;
        while ((c = getchar()) != '\n' && c != EOF) {}
    }
    destino[strcspn(destino, "\n")] = '\0';
}

/* Le um numero inteiro, repetindo ate o usuario digitar um valor valido */
int lerInteiro(const char *mensagem) {
    char linha[50];
    int valor;
    while (1) {
        lerTexto(mensagem, linha, sizeof(linha));
        if (sscanf(linha, "%d", &valor) == 1) {
            return valor;
        }
        printf("Valor invalido. Digite um numero inteiro.\n");
    }
}

/* Le um preco positivo e o devolve em centavos (inteiro), evitando erros de
   arredondamento do float. Aceita virgula ou ponto como separador decimal. */
int lerPreco(const char *mensagem) {
    char linha[50];
    float valor;
    int i;
    while (1) {
        lerTexto(mensagem, linha, sizeof(linha));
        for (i = 0; linha[i] != '\0'; i++) {
            if (linha[i] == ',')
                linha[i] = '.';
        }
        if (sscanf(linha, "%f", &valor) == 1 && valor > 0) {
            return (int)(valor * 100.0 + 0.5); /* reais -> centavos, arredondado */
        }
        printf("Preco invalido. Digite um valor maior que zero.\n");
    }
}

/* ---------- Arquivos ---------- */

/* RF11: acrescenta uma linha no historico: data/hora;acao;produto;filial;usuario */
void registrarHistorico(const char *acao, int idProduto, int filial,
                        const char *login) {
    FILE *arq = fopen(ARQ_HISTORICO, "a");
    time_t agora = time(NULL);
    char dataHora[20];
```

```
    if (arq == NULL) {
        printf("Aviso: nao foi possivel gravar o historico.\n");
        return;
    }
    strftime(dataHora, sizeof(dataHora), "%d/%m/%Y %H:%M", localtime(&agora));
    fprintf(arq, "%s;%s;%d;%s;%s\n", dataHora, acao, idProduto,
            (filial >= 0) ? FILIAIS[filial] : "Todas", login);
    fclose(arq);
}

/* Grava produtos e matriz de disponibilidade no arquivo texto.
   Formato de cada linha: id;categoria;nome;preco;alergenos;disp1;disp2;disp3 */
int salvarDados(void) {
    FILE *arq = fopen(ARQ_DADOS, "w");
    int i, j;

    if (arq == NULL) {
        printf("Erro ao salvar os dados.\n");
        return 0;
    }
    fprintf(arq, "%d\n", qtdProdutos);
    for (i = 0; i < qtdProdutos; i++) {
        fprintf(arq, "%d;%s;%s;%d;%s", produtos[i].id, produtos[i].categoria,
                produtos[i].nome, produtos[i].preco_cent, produtos[i].alergenos);
        for (j = 0; j < NUM_FILIAIS; j++) {
            fprintf(arq, ";%d", disponibilidade[i][j]);
        }
        fprintf(arq, "\n");
    }
    fclose(arq);
    return 1;
}

/* Le os dados do arquivo. Retorna 1 se conseguiu, 0 se o arquivo nao existe */
int carregarDados(void) {
    FILE *arq = fopen(ARQ_DADOS, "r");
    int i, j, total;

    if (arq == NULL) {
        return 0;
    }
    if (fscanf(arq, "%d\n", &total) != 1 || total < 0 || total > MAX_PRODUTOS) {
        fclose(arq);
        return 0;
    }
    for (i = 0; i < total; i++) {
        if (fscanf(arq, "%d;%29[^;];%59[^;];%d;%79[^;]", &produtos[i].id, produtos[i].categoria,
                   produtos[i].nome, &produtos[i].preco_cent, produtos[i].alergenos) != 5) {
            break;
        }

        for (j = 0; j < NUM_FILIAIS; j++) {
            fscanf(arq, ";%d", &disponibilidade[i][j]);
        }
        fscanf(arq, "\n");
    }
    qtdProdutos = i;
    fclose(arq);
    return 1;
}

/* ---------- Produtos ---------- */

/* Busca linear: retorna a posicao do produto no vetor ou -1 */
int buscarProdutoPorId(int id) {
    int i;
    for (i = 0; i < qtdProdutos; i++) {
        if (produtos[i].id == id) {
            return i;
        }
    }
```

```
    return -1;
}

/* Adiciona um produto no vetor, ativo em todas as filiais */
int adicionarProduto(const char *nome, const char *categoria, int precoCent,
                     const char *alergenos) {
    int j;
    if (qtdProdutos >= MAX_PRODUTOS) {
        return -1;
    }
    produtos[qtdProdutos].id =
        (qtdProdutos == 0) ? 1 : produtos[qtdProdutos - 1].id + 1;
    strncpy(produtos[qtdProdutos].nome, nome, TAM_NOME - 1);
    produtos[qtdProdutos].nome[TAM_NOME - 1] = '\0';
    strncpy(produtos[qtdProdutos].categoria, categoria, TAM_CATEGORIA - 1);
    produtos[qtdProdutos].categoria[TAM_CATEGORIA - 1] = '\0';
    produtos[qtdProdutos].preco_cent = precoCent;
    /* lista vazia vira "nenhum" para nao gerar campo em branco no arquivo */
    if (alergenos == NULL || alergenos[0] == '\0') {
        strcpy(produtos[qtdProdutos].alergenos, "nenhum");
    } else {
        strncpy(produtos[qtdProdutos].alergenos, alergenos, TAM_ALERGENOS - 1);
        produtos[qtdProdutos].alergenos[TAM_ALERGENOS - 1] = '\0';
    }
    for (j = 0; j < NUM_FILIAIS; j++) {
        disponibilidade[qtdProdutos][j] = 1;
    }
    qtdProdutos++;
    return produtos[qtdProdutos - 1].id;
}

/* Dados de exemplo usados na primeira execucao */
void carregarExemplo(void) {
    adicionarProduto("Burrata com tomates confitados", "Entradas", 6800, "Leite");
    adicionarProduto("Risoto de cogumelos", "Pratos principais", 9800, "Leite");
    adicionarProduto("Salmao em crosta de castanhas", "Pratos principais", 11200, "Peixe,Castanha-de-
caju");
    adicionarProduto("Petit gateau", "Sobremesas", 4200, "Leite,Ovos,Trigo");
    adicionarProduto("Chardonnay (taca)", "Vinhos", 4500, "");
    salvarDados();
}

/* RF05 (simplificado): cadastro de produto pelo administrador */
void cadastrarProduto(const char *login) {
    char nome[TAM_NOME], categoria[TAM_CATEGORIA], alergenos[TAM_ALERGENOS];
    int preco; /* em centavos (retorno de lerPreco) */
    int id;

    printf("\n--- Cadastrar produto ---\n");
    lerTexto("Nome: ", nome, sizeof(nome));
    lerTexto("Categoria: ", categoria, sizeof(categoria));
    if (strlen(nome) == 0 || strlen(categoria) == 0) {
        printf("Nome e categoria sao obrigatorios.\n");
        return;
    }
    lerTexto("Alergenos (separe por virgula, enter para nenhum): ", alergenos,
             sizeof(alergenos));
    if (strchr(nome, ';') != NULL || strchr(categoria, ';') != NULL ||
        strchr(alergenos, ';') != NULL) {
        printf("O caractere ';' nao e permitido.\n");
        return;
    }
    preco = lerPreco("Preco (R$): ");

    id = adicionarProduto(nome, categoria, preco, alergenos);
    if (id == -1) {
        printf("Limite de %d produtos atingido.\n", MAX_PRODUTOS);
        return;
    }
    salvarDados();
    registrarHistorico("CADASTRAR", id, -1, login);
    printf("Produto %d cadastrado e ativo em todas as filiais.\n", id);
```

```
}

/* Retorna 1 se a lista de alergenos contem o termo informado (ignora
   maiusculas/minusculas). Comparacao por substring, suficiente para a
   simulacao. Lista "nenhum" nunca casa com um filtro valido. */
int contemAlergeno(const char *lista, const char *termo) {
    char l[TAM_ALERGENOS], t[TAM_ALERGENOS];
    int i;
    if (termo == NULL || termo[0] == '\0') {
        return 0;
    }
    for (i = 0; lista[i] != '\0' && i < TAM_ALERGENOS - 1; i++) {
        l[i] = (char)tolower((unsigned char)lista[i]);
    }
    l[i] = '\0';
    for (i = 0; termo[i] != '\0' && i < TAM_ALERGENOS - 1; i++) {
        t[i] = (char)tolower((unsigned char)termo[i]);
    }
    t[i] = '\0';
    return strstr(l, t) != NULL;
}

/* RF07/RF13: mostra o cardapio de uma filial com os alergenos de cada item.
   somenteAtivos = 1 -> visao do cliente; 0 -> visao do gerente (com status).
   filtroAlergeno (se nao vazio) oculta, na visao do cliente, os pratos que
   contem o alergeno informado. */
void exibirCardapio(int filial, int somenteAtivos, const char *filtroAlergeno) {
    int i, exibidos = 0, ocultados = 0;


    printf("\n=== Sabor & Cia Bistro - %s ===\n", FILIAIS[filial]);
    for (i = 0; i < qtdProdutos; i++) {
        if (somenteAtivos && disponibilidade[i][filial] == 0) {
            continue; /* cliente nao ve itens esgotados */
        }
        if (somenteAtivos && contemAlergeno(produtos[i].alergenos, filtroAlergeno)) {
            ocultados++; /* cliente filtrou este alergeno */
            continue;
        }
        printf("[%3d] %-20s %-35s R$ %5d,%02d", produtos[i].id, produtos[i].categoria,
               produtos[i].nome, produtos[i].preco_cent / 100, produtos[i].preco_cent % 100);
        if (!somenteAtivos) {
            printf(" %s", disponibilidade[i][filial] ? "ATIVO" : "INATIVO");
        }
        printf(" | Alergenos: %s\n", produtos[i].alergenos);
        exibidos++;
    }
    if (exibidos == 0) {
        printf("Nenhum produto disponivel no momento.\n");
    }
    if (somenteAtivos && ocultados > 0) {
        printf("(%d item(ns) oculto(s) pelo filtro de alergeno \"%s\".)\n",
               ocultados, filtroAlergeno);
    }
    if (somenteAtivos) {
        /* RF13: aviso sobre alergenos */
        printf("\nAVISO: informacoes sobre alergenos sao de referencia.\n");
        printf("Em caso de alergia grave, confirme com o garcom.\n");
    }
}

/* Lista todos os produtos com a matriz de disponibilidade completa */
void listarTodos(void) {
    int i, j;


    printf("\n%-5s %-35s", "ID", "Produto");
    for (j = 0; j < NUM_FILIAIS; j++) {
        printf(" %-11s", FILIAIS[j]);
    }
    printf("\n");
    for (i = 0; i < qtdProdutos; i++) {
        printf("%-5d %-35s", produtos[i].id, produtos[i].nome);
        for (j = 0; j < NUM_FILIAIS; j++) {
```

```
            printf(" %-11s", disponibilidade[i][j] ? "ativo" : "INATIVO");
        }
        printf("\n");
    }
}

/* RF06: gerente ativa ou desativa um produto. A filial vem do login,
   entao o gerente nunca altera outra unidade. O novo estado e informado
   de forma explicita (1 = ativar, 2 = desativar), em vez de inverter o
   estado atual: repetir a mesma operacao nao desfaz a alteracao. */
void alterarDisponibilidade(int filial, const char *login) {
    int id, pos, opcao, novoEstado;

    exibirCardapio(filial, 0, "");
    id = lerInteiro("\nID do produto (0 para voltar): ");
    if (id == 0) {
        return;
    }
    pos = buscarProdutoPorId(id);
    if (pos == -1) {
        printf("Produto nao encontrado.\n");
        return;
    }

    do {
        opcao = lerInteiro("1 - Ativar 2 - Desativar: ");
    } while (opcao != 1 && opcao != 2);
    novoEstado = (opcao == 1) ? 1 : 0;

    if (disponibilidade[pos][filial] == novoEstado) {
        printf("'%s' ja estava %s. Nada foi alterado.\n", produtos[pos].nome,
               novoEstado ? "ATIVO" : "INATIVO");
        return;
    }

    disponibilidade[pos][filial] = novoEstado;
    salvarDados();
    registrarHistorico(novoEstado ? "ATIVAR" : "DESATIVAR", id, filial, login);
    printf("'%s' agora esta %s na filial %s.\n", produtos[pos].nome,
           novoEstado ? "ATIVO" : "INATIVO", FILIAIS[filial]);
}

/* RF12: quantos itens estao indisponiveis agora em cada filial */
void relatorioIndisponibilidade(void) {
    int i, j, contador;

    printf("\n--- Itens indisponiveis por filial ---\n");
    for (j = 0; j < NUM_FILIAIS; j++) {
        contador = 0;
        for (i = 0; i < qtdProdutos; i++) {
            if (disponibilidade[i][j] == 0) {
                contador++;
            }
        }
        printf("%-12s: %d de %d itens\n", FILIAIS[j], contador, qtdProdutos);
    }
}

/* Retorna o indice da filial pelo nome, ou -1 se nao existir */
int buscarFilialPorNome(const char *nome) {
    int j;
    for (j = 0; j < NUM_FILIAIS; j++) {
        if (strcmp(FILIAIS[j], nome) == 0) {
            return j;
        }
    }
    return -1;
}

/* RF12: frequencia de esgotamento. Le o historico.txt e conta, numa
```

```
   matriz produto x filial, quantas vezes cada item foi desativado. */
void relatorioFrequencia(void) {
    int frequencia[MAX_PRODUTOS][NUM_FILIAIS] = {{0}};
    char linha[200], acao[20], nomeFilial[30], login[20];
    int idProduto, pos, filial, i, j, total = 0;
    FILE *arq = fopen(ARQ_HISTORICO, "r");

    if (arq == NULL) {
        printf("Ainda nao ha historico de alteracoes.\n");
        return;
    }
    /* formato: data hora;acao;id;filial;login */
    while (fgets(linha, sizeof(linha), arq) != NULL) {
        if (sscanf(linha, "%*[^;];%19[^;];%d;%29[^;];%19s", acao, &idProduto,
                   nomeFilial, login) != 4) {
            continue; /* linha fora do formato: ignora */
        }
        if (strcmp(acao, "DESATIVAR") != 0) {
            continue;
        }
        pos = buscarProdutoPorId(idProduto);
        filial = buscarFilialPorNome(nomeFilial);
        if (pos != -1 && filial != -1) {
            frequencia[pos][filial]++;
            total++;
        }
    }
    fclose(arq);

    printf("\n--- Vezes que cada item esgotou, por filial ---\n");
    printf("%-35s", "Produto");
    for (j = 0; j < NUM_FILIAIS; j++) {
        printf(" %11s", FILIAIS[j]);
    }
    printf("\n");
    for (i = 0; i < qtdProdutos; i++) {
        printf("%-35.35s", produtos[i].nome);
        for (j = 0; j < NUM_FILIAIS; j++) {
            printf(" %11d", frequencia[i][j]);
        }
        printf("\n");
    }
    printf("Total de desativacoes registradas: %d\n", total);
}

/* ---------- Login e menus ---------- */

/* RF01: retorna o indice do usuario autenticado ou -1 apos 3 tentativas */
int autenticar(void) {
    char login[20], senha[20];
    int tentativa, i;

    printf("[SIMULACAO ACADEMICA] Usuarios de teste fixos no codigo; "
           "nao representa a seguranca do sistema real.\n");
    for (tentativa = 1; tentativa <= 3; tentativa++) {
        lerTexto("Login: ", login, sizeof(login));
        lerTexto("Senha: ", senha, sizeof(senha));
        for (i = 0; i < NUM_USUARIOS; i++) {
            if (strcmp(usuarios[i].login, login) == 0 &&
                strcmp(usuarios[i].senha, senha) == 0) {
                return i;
            }
        }
        printf("Login ou senha incorretos (%d/3).\n", tentativa);
    }
    return -1;
}

int escolherFilial(void) {
    int j, opcao;
    printf("\nFiliais:\n");
```

```
    for (j = 0; j < NUM_FILIAIS; j++) {
        printf("%d - %s\n", j + 1, FILIAIS[j]);
    }
    do {
        opcao = lerInteiro("Escolha a filial: ");
    } while (opcao < 1 || opcao > NUM_FILIAIS);
    return opcao - 1;
}

void menuAdmin(const char *login) {
    int opcao;
    do {
        printf("\n--- Painel do Administrador (matriz) ---\n");
        printf("1 - Cadastrar produto\n");
        printf("2 - Listar produtos e disponibilidade\n");
        printf("3 - Relatorio de indisponibilidade (agora)\n");
        printf("4 - Relatorio de frequencia de esgotamento\n");
        printf("0 - Sair\n");
        opcao = lerInteiro("Opcao: ");
        switch (opcao) {
        case 1:
            cadastrarProduto(login);
            break;
        case 2:
            listarTodos();
            break;
        case 3:
            relatorioIndisponibilidade();
            break;
        case 4:
            relatorioFrequencia();
            break;
        case 0:
            break;
        default:
            printf("Opcao invalida.\n");
        }
    } while (opcao != 0);
}

void menuGerente(int filial, const char *login) {
    int opcao;
    do {
        printf("\n--- Painel do Gerente - %s ---\n", FILIAIS[filial]);
        printf("1 - Ver cardapio da filial\n");
        printf("2 - Ativar/desativar produto\n");
        printf("0 - Sair\n");
        opcao = lerInteiro("Opcao: ");
        switch (opcao) {
        case 1:
            exibirCardapio(filial, 0, "");
            break;
        case 2:
            alterarDisponibilidade(filial, login);
            break;
        case 0:
            break;
        default:
            printf("Opcao invalida.\n");
        }
    } while (opcao != 0);
}

int main(void) {
    int opcao, indice, filialCliente;
    char filtro[TAM_ALERGENOS];

    if (!carregarDados()) {
        carregarExemplo();
    }

    do {
```

```
        printf("\n===== SABOR & CIA - CARDAPIO DIGITAL =====\n");
        printf("1 - Sou cliente (ver cardapio)\n");
        printf("2 - Acesso interno (login)\n");
        printf("0 - Encerrar\n");
        opcao = lerInteiro("Opcao: ");

        if (opcao == 1) {
            filialCliente = escolherFilial();
            lerTexto("Filtrar por alergeno (enter para ver tudo): ", filtro,
                     sizeof(filtro));
            exibirCardapio(filialCliente, 1, filtro);
        } else if (opcao == 2) {
            indice = autenticar();
            if (indice == -1) {
                printf("Acesso negado.\n");
            } else if (usuarios[indice].perfil == PERFIL_ADMIN) {
                menuAdmin(usuarios[indice].login);
            } else {
                menuGerente(usuarios[indice].filial, usuarios[indice].login);
            }
        } else if (opcao != 0) {
            printf("Opcao invalida.\n");
        }
    } while (opcao != 0);

    printf("Sistema encerrado.\n");
    return 0;
}
```

APÊNDICE B – SCRIPT DO MODELO FÍSICO EM MYSQL

```
-- ========================================================================
-- PIM II - Cardápio Digital Sabor & Cia Bistrô
-- Modelo físico (MySQL 8.0.16 ou superior; testado também em MariaDB)
-- ========================================================================

DROP DATABASE IF EXISTS sabor_cia;
CREATE DATABASE sabor_cia CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE sabor_cia;

-- ------------------------------------------------------------------------
-- 1. CRIAÇÃO DAS TABELAS (DDL)
-- ------------------------------------------------------------------------

CREATE TABLE filial (
    id_filial   INT AUTO_INCREMENT PRIMARY KEY,
    nome        VARCHAR(80)  NOT NULL,
    endereco    VARCHAR(150) NOT NULL,
    bairro      VARCHAR(60)  NOT NULL,
    url_qrcode  VARCHAR(255) NOT NULL UNIQUE
);

-- Administrador e Gerente ficam na mesma tabela (herança de tabela única).
-- Regra: gerente sempre tem filial; administrador (matriz) nunca tem.
CREATE TABLE usuario (
    id_usuario  INT AUTO_INCREMENT PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    email       VARCHAR(120) NOT NULL UNIQUE,
    senha_hash  CHAR(60)     NOT NULL,          -- RNF04: hash bcrypt, nunca a senha
    perfil      ENUM('ADMIN','GERENTE') NOT NULL,
    id_filial   INT NULL,
    CONSTRAINT fk_usuario_filial FOREIGN KEY (id_filial)
        REFERENCES filial (id_filial),
    CONSTRAINT ck_usuario_perfil CHECK (
        (perfil = 'ADMIN'   AND id_filial IS NULL) OR
        (perfil = 'GERENTE' AND id_filial IS NOT NULL)
    )
);

-- RF10: cada maquininha tem um token de API vinculado a uma filial.
-- A API usa esse vínculo para saber de qual unidade é o catálogo.
CREATE TABLE dispositivo (
    id_dispositivo INT AUTO_INCREMENT PRIMARY KEY,
    id_filial      INT NOT NULL,
    descricao      VARCHAR(60) NOT NULL,
    token_hash     CHAR(64) NOT NULL UNIQUE,     -- SHA-256 do token, nunca o token
    ativo          BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_dispositivo_filial FOREIGN KEY (id_filial)
        REFERENCES filial (id_filial)
);

CREATE TABLE categoria (
    id_categoria    INT AUTO_INCREMENT PRIMARY KEY,
    nome_pt         VARCHAR(60) NOT NULL UNIQUE,
    nome_en         VARCHAR(60) NOT NULL,
    ordem_exibicao  INT NOT NULL
);

CREATE TABLE produto (
    id_produto       INT AUTO_INCREMENT PRIMARY KEY,
    id_categoria     INT NOT NULL,
    nome_pt          VARCHAR(100) NOT NULL,
    nome_en          VARCHAR(100) NOT NULL,
    descricao_pt     TEXT,
    descricao_en     TEXT,
    ingredientes_pt  TEXT,   -- texto descritivo exibido ao cliente (não é consultado)
    ingredientes_en  TEXT,
    preco            DECIMAL(10,2) NOT NULL,
    url_foto         VARCHAR(255),
```

```
    CONSTRAINT fk_produto_categoria FOREIGN KEY (id_categoria)
        REFERENCES categoria (id_categoria),
    CONSTRAINT ck_produto_preco CHECK (preco > 0)
);

-- Nomes seguem a lista de alergênicos da Anvisa (ex.: "Leite", não "Lactose")
CREATE TABLE alergeno (
    id_alergeno  INT AUTO_INCREMENT PRIMARY KEY,
    nome_pt      VARCHAR(50) NOT NULL UNIQUE,
    nome_en      VARCHAR(50) NOT NULL
);

-- N:N produto x alérgeno
CREATE TABLE produto_alergeno (
    id_produto   INT NOT NULL,
    id_alergeno  INT NOT NULL,
    PRIMARY KEY (id_produto, id_alergeno),
    CONSTRAINT fk_pa_produto  FOREIGN KEY (id_produto)
        REFERENCES produto (id_produto) ON DELETE CASCADE,
    CONSTRAINT fk_pa_alergeno FOREIGN KEY (id_alergeno)
        REFERENCES alergeno (id_alergeno)
);

-- N:N auto-relacionamento: prato harmoniza com vinho
CREATE TABLE harmonizacao (
    id_prato  INT NOT NULL,
    id_vinho  INT NOT NULL,
    PRIMARY KEY (id_prato, id_vinho),
    CONSTRAINT fk_harm_prato FOREIGN KEY (id_prato)
        REFERENCES produto (id_produto) ON DELETE CASCADE,
    CONSTRAINT fk_harm_vinho FOREIGN KEY (id_vinho)
        REFERENCES produto (id_produto) ON DELETE CASCADE,
    CONSTRAINT ck_harm_diferente CHECK (id_prato <> id_vinho)
);

-- N:N produto x filial com atributos: coração do sistema
CREATE TABLE disponibilidade (
    id_produto     INT NOT NULL,
    id_filial      INT NOT NULL,
    ativo          BOOLEAN  NOT NULL DEFAULT TRUE,
    atualizado_em  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
                   ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id_produto, id_filial),
    CONSTRAINT fk_disp_produto FOREIGN KEY (id_produto)
        REFERENCES produto (id_produto) ON DELETE CASCADE,
    CONSTRAINT fk_disp_filial  FOREIGN KEY (id_filial)
        REFERENCES filial (id_filial) ON DELETE CASCADE
);

-- RF11: histórico de alterações de disponibilidade.
-- Sem ON DELETE CASCADE: o histórico precisa sobreviver ao produto.
CREATE TABLE historico_disponibilidade (
    id_historico  BIGINT AUTO_INCREMENT PRIMARY KEY,
    id_produto    INT NOT NULL,
    id_filial     INT NOT NULL,
    id_usuario    INT NOT NULL,
    acao          ENUM('ATIVAR','DESATIVAR') NOT NULL,
    data_hora     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_hist_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
    CONSTRAINT fk_hist_filial  FOREIGN KEY (id_filial)  REFERENCES filial (id_filial),
    CONSTRAINT fk_hist_usuario FOREIGN KEY (id_usuario) REFERENCES usuario (id_usuario)
);

-- Índices para as consultas mais frequentes
CREATE INDEX idx_disp_filial_ativo ON disponibilidade (id_filial, ativo);
CREATE INDEX idx_hist_filial_data  ON historico_disponibilidade (id_filial, data_hora);

-- Regra de negócio que um CHECK não alcança (depende de outra tabela):
-- em harmonizacao, id_vinho precisa ser da categoria "Vinhos" e id_prato não.
-- A regra vale na inclusão e na alteração, por isso há dois triggers.
DELIMITER //
```

```
CREATE TRIGGER trg_harmonizacao_valida_ins
BEFORE INSERT ON harmonizacao
FOR EACH ROW
BEGIN
    IF (SELECT c.nome_pt FROM produto p JOIN categoria c ON c.id_categoria = p.id_categoria
        WHERE p.id_produto = NEW.id_vinho) <> 'Vinhos'
       OR (SELECT c.nome_pt FROM produto p JOIN categoria c ON c.id_categoria = p.id_categoria
           WHERE p.id_produto = NEW.id_prato) = 'Vinhos' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Harmonizacao deve ligar um prato a um vinho';
    END IF;
END //

CREATE TRIGGER trg_harmonizacao_valida_upd
BEFORE UPDATE ON harmonizacao
FOR EACH ROW
BEGIN
    IF (SELECT c.nome_pt FROM produto p JOIN categoria c ON c.id_categoria = p.id_categoria
        WHERE p.id_produto = NEW.id_vinho) <> 'Vinhos'
       OR (SELECT c.nome_pt FROM produto p JOIN categoria c ON c.id_categoria = p.id_categoria
           WHERE p.id_produto = NEW.id_prato) = 'Vinhos' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Harmonizacao deve ligar um prato a um vinho';
    END IF;
END //
DELIMITER ;

-- ---------------------------------------------------------------------
-- 2. CARGA DE EXEMPLO (DML)
-- ---------------------------------------------------------------------

INSERT INTO filial (nome, endereco, bairro, url_qrcode) VALUES
('Sabor & Cia Bistrô Jardins',   'Rua Exemplo, 100', 'Jardins',
'https://cardapio.saborecia.com.br/f/1'),
('Sabor & Cia Bistrô Itaim',     'Rua Exemplo, 200', 'Itaim Bibi',
'https://cardapio.saborecia.com.br/f/2'),
('Sabor & Cia Bistrô Pinheiros', 'Rua Exemplo, 300', 'Pinheiros',
'https://cardapio.saborecia.com.br/f/3');

INSERT INTO usuario (nome, email, senha_hash, perfil, id_filial) VALUES
('Ana Matriz',    'ana@saborecia.com.br',
'$2b$12$abcdefghijklmnopqrstuuABCDEFGHIJKLMNOPQRSTUVWXYZ01234', 'ADMIN',   NULL),
('Bruno Jardins', 'bruno@saborecia.com.br',
'$2b$12$abcdefghijklmnopqrstuuABCDEFGHIJKLMNOPQRSTUVWXYZ01235', 'GERENTE', 1),
('Carla Itaim',   'carla@saborecia.com.br',
'$2b$12$abcdefghijklmnopqrstuuABCDEFGHIJKLMNOPQRSTUVWXYZ01236', 'GERENTE', 2);

-- Tokens de exemplo (valores de SHA-256 fictícios)
INSERT INTO dispositivo (id_filial, descricao, token_hash) VALUES
(1, 'Maquininha salão 01', SHA2('token-exemplo-jardins-01', 256)),
(2, 'Maquininha salão 01', SHA2('token-exemplo-itaim-01', 256));

INSERT INTO categoria (nome_pt, nome_en, ordem_exibicao) VALUES
('Entradas', 'Starters', 1),
('Pratos principais', 'Main courses', 2),
('Sobremesas', 'Desserts', 3),
('Vinhos', 'Wines', 4);

INSERT INTO produto (id_categoria, nome_pt, nome_en, ingredientes_pt, ingredientes_en, preco) VALUES
(1, 'Burrata com tomates confitados', 'Burrata with confit tomatoes',
    'burrata, tomate, manjericão, azeite', 'burrata, tomato, basil, olive oil', 68.00),
(2, 'Risoto de cogumelos', 'Mushroom risotto',
    'arroz arbóreo, cogumelos, parmesão, manteiga', 'arborio rice, mushrooms, parmesan, butter',
98.00),
(2, 'Salmão em crosta de castanhas', 'Nut-crusted salmon',
    'salmão, castanha-de-caju, purê de mandioquinha', 'salmon, cashew nuts, arracacha purée', 112.00),
(3, 'Petit gâteau', 'Petit gâteau',
    'chocolate, ovos, farinha de trigo, sorvete', 'chocolate, eggs, wheat flour, ice cream', 42.00),
(4, 'Chardonnay (taça)', 'Chardonnay (glass)', NULL, NULL, 45.00);
```

```
INSERT INTO alergeno (nome_pt, nome_en) VALUES
('Leite', 'Milk'), ('Trigo', 'Wheat'), ('Castanha-de-caju', 'Cashew'),
('Ovos', 'Eggs'), ('Peixe', 'Fish');


INSERT INTO produto_alergeno (id_produto, id_alergeno) VALUES
(1,1), (2,1), (3,3), (3,5), (4,1), (4,2), (4,4);


INSERT INTO harmonizacao (id_prato, id_vinho) VALUES (2,5), (3,5);


-- Todo produto começa disponível em todas as filiais
INSERT INTO disponibilidade (id_produto, id_filial)
SELECT p.id_produto, f.id_filial FROM produto p CROSS JOIN filial f;


-- ---------------------------------------------------------------------
-- 3. PRINCIPAIS COMANDOS DO SISTEMA
-- ---------------------------------------------------------------------


-- RF06 + RF11: o gerente Bruno (id_usuario 2, filial 1) desativa o risoto.
-- A filial vem do cadastro do usuário, não de um campo digitado. Atenção:
-- quem garante que @usuario é o gerente autenticado é a API (pelo login);
-- o banco apenas aplica a filial desse usuário. Se @usuario for um ADMIN
-- (sem filial), nenhuma linha é alterada.
-- O valor é explícito (FALSE), então repetir o comando não desfaz nada.
SET @usuario = 2, @produto = 2;
START TRANSACTION;
UPDATE disponibilidade d
JOIN usuario u ON u.id_usuario = @usuario AND u.perfil = 'GERENTE'
SET d.ativo = FALSE
WHERE d.id_produto = @produto
  AND d.id_filial = u.id_filial
  AND d.ativo = TRUE;             -- só registra se o estado realmente mudar
SET @alterou = ROW_COUNT();     -- lido logo após o UPDATE
INSERT INTO historico_disponibilidade (id_produto, id_filial, id_usuario, acao)
SELECT @produto, u.id_filial, u.id_usuario, 'DESATIVAR'
FROM usuario u
WHERE u.id_usuario = @usuario AND @alterou > 0;
COMMIT;


-- Mais alguns eventos de exemplo para o relatório de frequência (RF12)
INSERT INTO historico_disponibilidade (id_produto, id_filial, id_usuario, acao, data_hora) VALUES
(3, 2, 3, 'DESATIVAR', '2026-09-20 20:15:00'),
(3, 2, 3, 'ATIVAR',    '2026-09-21 11:00:00'),
(3, 2, 3, 'DESATIVAR', '2026-09-26 21:05:00'),
(3, 2, 3, 'ATIVAR',    '2026-09-27 11:00:00'),
(2, 1, 2, 'DESATIVAR', '2026-09-22 13:40:00'),
(2, 1, 2, 'ATIVAR',    '2026-09-23 11:00:00');


-- RF07 / RF10: cardápio disponível da filial 1 (cliente e maquininha)
SELECT c.nome_pt AS categoria, p.nome_pt AS produto, p.preco
FROM disponibilidade d
JOIN produto   p ON p.id_produto   = d.id_produto
JOIN categoria c ON c.id_categoria = p.id_categoria
WHERE d.id_filial = 1 AND d.ativo = TRUE
ORDER BY c.ordem_exibicao, p.nome_pt;


-- RF10: a maquininha se identifica pelo token; a filial vem do dispositivo
SELECT p.id_produto, p.nome_pt, p.preco
FROM dispositivo disp
JOIN disponibilidade d ON d.id_filial = disp.id_filial AND d.ativo = TRUE
JOIN produto p ON p.id_produto = d.id_produto
WHERE disp.token_hash = SHA2('token-exemplo-jardins-01', 256) AND disp.ativo = TRUE;


-- RF09: cardápio da filial 1 sem leite (para quem tem alergia ao leite)
SELECT p.nome_pt, p.preco
FROM disponibilidade d
JOIN produto p ON p.id_produto = d.id_produto
WHERE d.id_filial = 1 AND d.ativo = TRUE
  AND NOT EXISTS (
      SELECT 1 FROM produto_alergeno pa
      JOIN alergeno a ON a.id_alergeno = pa.id_alergeno
```

```
      WHERE pa.id_produto = p.id_produto AND a.nome_pt = 'Leite'
  );

-- Harmonizações sugeridas
SELECT prato.nome_pt AS prato, vinho.nome_pt AS vinho_sugerido
FROM harmonizacao h
JOIN produto prato ON prato.id_produto = h.id_prato
JOIN produto vinho ON vinho.id_produto = h.id_vinho;


-- RF12: itens indisponíveis agora, por filial (inclui filiais com zero)
SELECT f.nome, COALESCE(SUM(d.ativo = FALSE), 0) AS itens_indisponiveis
FROM filial f
LEFT JOIN disponibilidade d ON d.id_filial = f.id_filial
GROUP BY f.id_filial, f.nome
ORDER BY f.id_filial;


-- RF12: quantas vezes cada produto esgotou em cada filial nos últimos 30 dias
SELECT f.nome AS filial, p.nome_pt AS produto, COUNT(*) AS vezes_esgotado
FROM historico_disponibilidade h
JOIN filial  f ON f.id_filial  = h.id_filial
JOIN produto p ON p.id_produto = h.id_produto
WHERE h.acao = 'DESATIVAR'
  AND h.data_hora >= NOW() - INTERVAL 30 DAY
GROUP BY f.nome, p.nome_pt
ORDER BY vezes_esgotado DESC, f.nome;
```

APÊNDICE C – PSEUDOCÓDIGO DAS PRINCIPAIS FUNCIONALIDADES

```
PIM II - Sabor & Cia Bistro
Pseudocodigo das principais funcionalidades (estilo Portugol)

-----------------------------------------------------------------
Algoritmo 1: ExibirCardapio (RF07 e RF13)
-----------------------------------------------------------------
// "filtroAlergeno" (opcional) oculta, na visao do cliente, os pratos que
// contem o alergeno informado. Vazio = sem filtro. Usa ContemAlergeno (Alg. 6).
funcao ExibirCardapio(filial: inteiro, somenteAtivos: logico, filtroAlergeno: caractere)
var
    i, exibidos, ocultados: inteiro
inicio
    exibidos <- 0
    ocultados <- 0
    escreva("=== Sabor & Cia Bistro - ", FILIAIS[filial], " ===")
    para i de 0 ate qtdProdutos - 1 faca
        // o cliente so ve itens ativos; o gerente ve todos, com o status
        se (somenteAtivos = falso) ou (disponibilidade[i][filial] = 1) entao
            // na visao do cliente, oculta os itens que contem o alergeno filtrado
            se (somenteAtivos = verdadeiro) e ContemAlergeno(produtos[i].alergenos, filtroAlergeno)
entao
                ocultados <- ocultados + 1
            senao
                escreva(produtos[i].id, produtos[i].categoria,
                    produtos[i].nome, produtos[i].preco)
                se somenteAtivos = falso entao
                    se disponibilidade[i][filial] = 1 entao
                        escreva(" ATIVO")
                    senao
                        escreva(" INATIVO")
                    fimse
                fimse
                escreva(" | Alergenos: ", produtos[i].alergenos)
                exibidos <- exibidos + 1
            fimse
        fimse
    fimpara
    se exibidos = 0 entao
        escreva("Nenhum produto disponivel no momento.")
    fimse
    se (somenteAtivos = verdadeiro) e (ocultados > 0) entao
        escreva("(", ocultados, " item(ns) oculto(s) pelo filtro de alergeno \"", filtroAlergeno,
"\".)")
    fimse
    se somenteAtivos = verdadeiro entao
        escreva("AVISO: informacoes sobre alergenos sao de referencia.")
        escreva("Em caso de alergia grave, confirme com o garcom.")
    fimse
fimfuncao

-----------------------------------------------------------------
Algoritmo 2: AlterarDisponibilidade (RF06 e RF11)
-----------------------------------------------------------------
funcao AlterarDisponibilidade(filial: inteiro, login: caractere)
// "filial" vem do login do gerente; ele nunca informa outra unidade.
// O novo estado e escolhido de forma explicita: repetir a operacao
// nao desfaz a alteracao.
var
    id, pos, opcao, novoEstado: inteiro
    acao: caractere
inicio
    ExibirCardapio(filial, falso, "")   // visao do gerente: sem filtro de alergeno
    escreva("ID do produto (0 para voltar): ")
    leia(id)
    se id = 0 entao
        retorne
    fimse
    pos <- BuscarProdutoPorId(id)
    se pos = -1 entao
        escreva("Produto nao encontrado.")
```

```
        retorne
    fimse
    repita
        escreva("1 - Ativar 2 - Desativar: ")
        leia(opcao)
    ate (opcao = 1) ou (opcao = 2)
    se opcao = 1 entao
        novoEstado <- 1
        acao <- "ATIVAR"
    senao
        novoEstado <- 0
        acao <- "DESATIVAR"
    fimse
    se disponibilidade[pos][filial] = novoEstado entao
        escreva("O produto ja estava nesse estado. Nada foi alterado.")
        retorne
    fimse
    disponibilidade[pos][filial] <- novoEstado
    SalvarDados()                                  // grava cardapio.txt
    RegistrarHistorico(acao, id, filial, login)    // acrescenta em historico.txt
    escreva("Alteracao registrada.")
fimfuncao

-----------------------------------------------------------------
Algoritmo 3: BuscarProdutoPorId (busca linear)
-----------------------------------------------------------------
funcao BuscarProdutoPorId(id: inteiro): inteiro
var
    i: inteiro
inicio
    para i de 0 ate qtdProdutos - 1 faca
        se produtos[i].id = id entao
            retorne i
        fimse
    fimpara
    retorne -1
fimfuncao

-----------------------------------------------------------------
Algoritmo 4: Autenticar (RF01, ate 3 tentativas)
-----------------------------------------------------------------
funcao Autenticar(): inteiro
var
    tentativa, i: inteiro
    login, senha: caractere
inicio
    para tentativa de 1 ate 3 faca
        leia(login)
        leia(senha)
        para i de 0 ate NUM_USUARIOS - 1 faca
            se (usuarios[i].login = login) e (usuarios[i].senha = senha) entao
                retorne i
            fimse
        fimpara
        escreva("Login ou senha incorretos (", tentativa, "/3).")
    fimpara
    retorne -1
fimfuncao

-----------------------------------------------------------------
Algoritmo 5: RelatorioFrequencia (RF12)
-----------------------------------------------------------------
funcao RelatorioFrequencia()
// conta, numa matriz produto x filial, quantas vezes cada item foi
// desativado, lendo o arquivo historico.txt linha a linha
var
    frequencia: vetor[0..MAX_PRODUTOS-1, 0..NUM_FILIAIS-1] de inteiro
    dataHora, acao, nomeFilial, login: caractere
    idProduto, pos, filial, i, j: inteiro
    arq: arquivo
inicio
    para i de 0 ate qtdProdutos - 1 faca
        para j de 0 ate NUM_FILIAIS - 1 faca
```

```
            frequencia[i][j] <- 0
        fimpara
    fimpara
    abra(arq, "historico.txt", leitura)
    se arq = nulo entao
        escreva("Ainda nao ha historico de alteracoes.")
        retorne
    fimse
    enquanto nao fimarquivo(arq) faca
        leia(arq, dataHora, acao, idProduto, nomeFilial, login)
        se acao = "DESATIVAR" entao
            pos <- BuscarProdutoPorId(idProduto)
            filial <- BuscarFilialPorNome(nomeFilial)
            se (pos <> -1) e (filial <> -1) entao
                frequencia[pos][filial] <- frequencia[pos][filial] + 1
            fimse
        fimse
    fimenquanto
    feche(arq)
    para i de 0 ate qtdProdutos - 1 faca
        escreva(produtos[i].nome)
        para j de 0 ate NUM_FILIAIS - 1 faca
            escreva(frequencia[i][j])
        fimpara
    fimpara
fimfuncao


-----------------------------------------------------------------
Algoritmo 6: ContemAlergeno (apoio ao filtro do RF13)
-----------------------------------------------------------------
funcao ContemAlergeno(lista: caractere, termo: caractere): logico
// retorna verdadeiro se "termo" aparece em "lista" (lista separada por
// virgula), ignorando maiusculas/minusculas. Termo vazio retorna falso.
var
    listaMin, termoMin: caractere
inicio
    se termo = "" entao
        retorne falso
    fimse
    listaMin <- ParaMinusculas(lista)
    termoMin <- ParaMinusculas(termo)
    se termoMin e subcadeia de listaMin entao
        retorne verdadeiro
    senao
        retorne falso
    fimse
fimfuncao
```